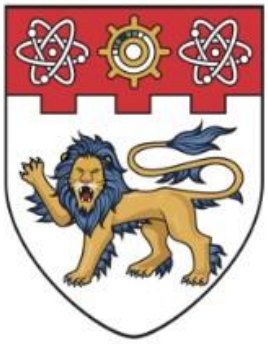
A3069-251



NANYANG TECHNOLOGICAL UNIVERSITY
SINGAPORE

PhytoLens – An AI assisted mobile application to identify plants/ animals using the camera.

Submitted By: Luqman Nurhakim Bin Sairi

Matriculation Number: U2322138D

Supervisor's: A/P Law Choi Look

Examiner: Dr Ge Tong

School of Electrical & Electronic Engineering

A final year project report presented to Nanyang Technological University in partial fulfilment of the requirements for the Degree of Bachelor of Engineering

2025/2026

Table of Contents

Abstract 5

Acknowledgement 6

Acronyms 7

List of Figures 8

Chapter 1: Introduction 10

1.1 – Background and Motivation 10

1.2 – Objectives and Scope 11

1.3 – Timeline of the project 12

Chapter 2 – Literature Review 13

2.1 – Researching on known mobile development frameworks 13

2.1.1 – React Native 13

2.1.2 – Flutter 13

2.1.3 Mobile Angular UI 14

2.1.4 – Ionic 14

2.1.5 – Xamarin 14

2.1.6 – Adobe PhoneGap 15

2.1.7 - Appcelerator Titanium 15

2.1.8 - Framework 7 15

2.1.9 - JQuery Mobile 16

2.1.10 – NativeScript 16

2.1.11 – Android Software Development Kit (SDK) 17

2.2 – Discovering the various AI API models 17

2.2.1 – DALL E 17

2.2.2 – GPT 17

2.2.3 – Gemini 17

2.3 – Cloud-Based Database for PhytoLens 18

2.3.1 – Firebase 18

2.3.2 – SQLite 18

2.3.3 – Realm 18

2.3.4 – Couchbase Mobile 18

2.3.5 – Amazon DynamoDB 18
2.3.6 – MongoDB Realm 19
2.3.7 - PostgreSQL 19
2.4 – Selecting the tools for development 19
2.4.1 – Choosing the Framework 19
2.4.2 – Choosing of the Database 19
2.4.3 – Choosing of the AI API model 19
Chapter 3 – Development and Implementation 20
3.1 – Front-End Development 20
3.1.1 – PhytoLens Landing 20
3.1.2 – Capture screen 23
3.1.3 – Analyse image screen 27
3.1.4 – Results of analysing image 28
3.1.5 – Analysis History 30
3.1.6 – History Details 32
3.1.7 – Profile screen 33
3.1.8 – Admin login 36
3.1.9 – Search 38
3.1.10 – Types of plants 40
3.1.11 - Photosynthesis 40
3.1.12 – Fun Facts 42
3.1.13 – Articles 43
3.1.14 – Support / FAQ 44
3.1.15 – Forums 45
3.2 – Back-End Development 46
3.2.1 – Creation of Project 46
3.2.2 – Setting up the Authentication 47
3.2.3 – Firestore Database setup 47
3.2.4 – Firebase Storage 48
3.2.5 – Comment Images 48
3.1.6 – Thread Post Images 49
Chapter 4 – Testing and Results 50
4.1 – Testcase 1: Dynamic Identification 50

4.2 – Testcase 2: Identification History Filtering....53
4.3 – Testcase 3: Lifecycle resetting....55
4.4 – Testcase 4: Cloud Routing....58
4.5 – Testcase 5: Data Privacy....60
4.6 – Testcase 6: Admin Access....63
Chapter 5 – Conclusion & Future work....66
5.1 – Conclusion....66
5.2 – Recommendation in Future Work....66
Reflection on Learning Outcome Attainment....68
References....69

Abstract

Venturing out in the wilderness is something that a lot of people do to clear their minds and be closer to nature. Encountering an unknown species of organisms is normal for people who are out there for leisure. Touching something not known to us is often not life threatening but it is better to ensure its safe rather than assume it. This project introduces PhytoLens, an AI assisted mobile app developed for people who loves to explore the wilderness or those who love to do botany to help identify plants to see whether it is poisonous or whether it is edible using the device's built-in camera.

PhytoLens allows the user to take a photo of a plant or animal using the device's camera or select an existing image from the photo gallery to identify the plant or animal from the photo taken. After the image has been analysed by the AI, it will display out the layman's name, scientific name, whether its poisonous or venomous, whether there are recipes if its not poisonous or venomous and fun facts about the analysed plant or animal. This especially helps the people in the wilderness where they might encounter something unknown to them and this app will help to identify the thing that they encountered easily without surfing the net to find out what it is. This helps to cut down precious time wasted if they are poisoned by the plant or insect. This app is developed using Android Studio with Firebase for the backend management so, it is only compatible with Android currently.

Acknowledgement

I would like to extend my heartfelt thanks to A/P Law Choi Look for his unwavering guidance, support and encouragement throughout the FYP. This project would not have been running smoothly without his insightful feedback and thoughtful recommendations. His drive to start the project early in the semester helped me be ahead of most of my peers.

I would also like to express my sincere gratitude to my friends and family who had supported me throughout the project thus far by giving constant feedback on how I can improve on the application and what additional features I can add on in the future.

All in all, the people around me had a significant play in helping me get this project to this stage which is satisfactory for me such that most of the functions are working. The upmost support from my parents for helping me and motivating me daily helped me to keep pushing to complete this project since there were many hurdles along the way.

Acronyms

NTU	Nanyang Technological University
EEE	Electrical and Electronic Engineering
UI	User Interface
UX	User Experience
DB	Database
RDBMS	Relational Database Management System
FAQ	Frequently Asked Questions
R&D	Research & Development
LLM	Large Language Model
AR	Augmented Reality

List of Figures

Figure 1: Landing Page of PhytoLens 20
Figure 2: PhytoLens landing screen + Navigation button redirections 22
Figure 3: Choosing to identify plants or insects 23
Figure 4: Capture screen 23
Figure 5: startCamera method to use the camera function 24
Figure 6: Image gallery (Cropped out the bottom to only show relevant items(plants)) 24
Figure 7:Snapped image of a plant 25
Figure 8: Tips button 26
Figure 9: Analysing of image 27
Figure 10: Results after analysing image 28
Figure 11: Deep Dive analysis 29
Figure 12: Analysis History screen (plants) 30
Figure 13: Analysis History screen (insects) 31
Figure 14: History without more info 32
Figure 15: History with more info 32
Figure 16: Profile Page 33
Figure 17: Profile Page Plants Filter 34
Figure 18: Profile Page Insects Filter 34
Figure 19: Login and Create New Account screens with their respective forms 35
Figure 20: Admin account 36
Figure 21: Normal User Account 36
Figure 22: Guest User Profile 37
Figure 23: Search function in mainpage 38
Figure 24: Search result 39
Figure 25: Types of Plants 40
Figure 26: Photosynthesis 41
Figure 27: Fun Facts 42
Figure 28: Articles 43
Figure 29: Support/FAQ and email 44
Figure 30: Forum Topic 45
Figure 31: Forum Page 45
Figure 32: Posting Comments 45
Figure 33: Admin Delete Thread 45
Figure 34: Start New Thread 45
Figure 35: Firebase PhytoLens Project Overview 46
Figure 36: Choosing of Firebase functions for PhytoLens 46
Figure 37: Authentication setup 47
Figure 38: Firestore Database 47
Figure 39: Firebase Storage analysis_images 48
Figure 40: Segregation between registered users and guests' images 48
Figure 41: Storage 48

Figure 42: Inside Comment Images folder 49
Figure 43: Inside Post Images folder 49
Figure 44: TC1 Img1 50
Figure 45: TC1 Img2 51
Figure 46: TC1 Img3 51
Figure 47: TC1 Img4 52
Figure 48:TC1 Img5 52
Figure 49: TC2 Img1 53
Figure 50:TC2 Img2 54
Figure 51:TC2 Img3 54
Figure 52: TC3 Img1 and 2.....55
Figure 53: TC3 Img3 56
Figure 54: TC3 Img4 57
Figure 55: TC3 Img5 57
Figure 56: TC4 Img1 and 2.....58
Figure 57: TC4 Img3 59
Figure 58: TC4 Img4 and 5.....59
Figure 59: TC5 Img1 60
Figure 60: TC5 Img2 and 3.....61
Figure 61: TC5 Img4 61
Figure 62: TC5 Img5 62
Figure 63: TC6 Img1 63
Figure 64: TC6 Img2 64

Chapter 1: Introduction

1.1 – Background and Motivation

Spending time outdoors, whether hiking, foraging or studying the local biodiversity often brings people to encounter species that they cannot readily recognise and identify. While curiosity is a great way of learning but the quote "curiosity killed the cat" really comes into play here as misidentifying plants or animals can carry real risks especially with toxic plants, venomous animals or species that look alike that are difficult for non-experts to distinguish. Although information about how to identify species are abundant online, it is fragmented across multiple websites and forums, field guides and encyclopaedic resources. In time-sensitive situations, searching for information across multiple sources individually is impractical. At the same time, the advancements of technology in mobile devices and AI had created an opportunity to deliver accurate and context-rich identification off the fly at the point of need.

Existing tools for visual identification had demonstrated the feasibility of recognising species from photos. However, they often prioritise taxonomic accuracy rather than safety-centered guidance and layman's friendly explanations. Users still need to piece together information like toxicity, edibility and handling precautions from multiple sources. Still this content does not guarantee clarity and reliability.

PhytoLens will address this gap by providing an AI-Assisted mobile application that allows for the identification of species with concise and safety-relevant information. PhytoLens uses the device's camera or an image from the device's gallery to analyse a photo. It would then return a common and scientific names together with the toxicity or venom status and if it is safe, practical information such as culinary uses and recipes would be displayed. On top of that, fun facts would be generated too for supported learning. So, by consolidating these outputs into a single and intuitive interface, it allows hikers and nature enthusiasts to make quicker and more informed judgements without the need of sifting through information from various sources online. PhytoLens is developed in Android Studio with Firebase for data and user management.

The motivation in the development of PhytoLens is for safety and education. On the safety side, hikers and casual foragers who frequently encounter unfamiliar flora and fauna and

timely and simple to understand guidance can reduce risky interactions and support choices. For the education side, presenting the identifications alongside explanations can turn encounters into learning opportunities. This leads to encouraging responsible interactions with nature and reinforcing botanical and ecological literacy. PhytoLens is designed to focus on clarity - generating easy to understand summaries, toxicity indicators and actionable notes to reduce the cognitive load on non-experts.

Developing PhytoLens has multiple challenges like handling the image quality and lighting and being able to distinguish visually similar species within the constraints of mobile hardware and connectivity. By using existing AI models, safety-focused content can be delivered through the Android and Firebase stack. PhytoLens is solely intended as a decision support rather than a definitive tool where users are always encouraged to exercise caution and avoid consuming or handling wild organisms based on the information generated as AI is not perfect yet.

PhytoLens is motivated by a clear, real-world need which is fast, trustworthy, and understandable species identification that prioritises the user’s safety while fostering curiosity and learning. It is the melting pot of the advancements in AI and the reach of mobile devices to make responsible nature exploration more accessible to everyone.

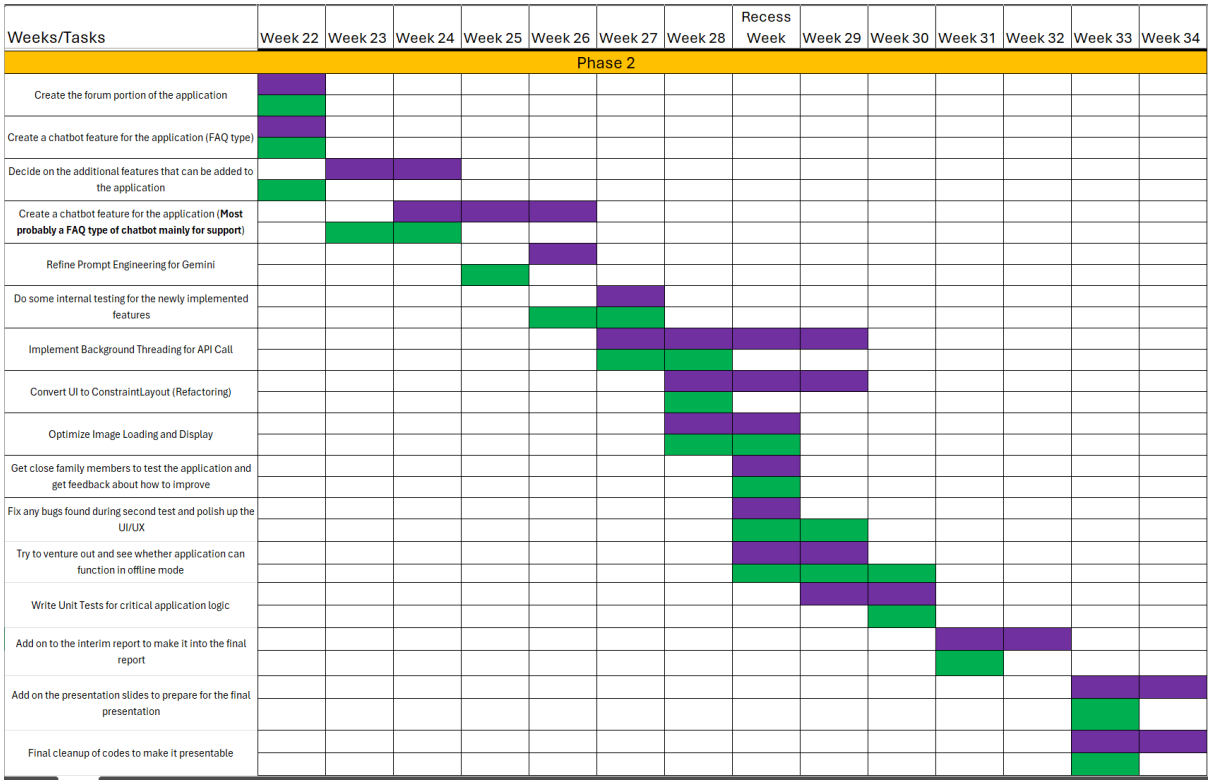
1.2 – Objectives and Scope

This project aims to develop a mobile application to identify plants/animals and provide the user with useful information. The mobile app shall use the smartphone camera to capture a picture of the object and shall utilize existing generative AI model where possible. This app target at hikers as there are many poisonous plants in the wild that many do not know about. This can help to identify any poisonous plants along with their trails. The application by default needs internet access to connect to the AI and database. If time permits, this project shall also explore some of the functions like identifying objects in offline mode.

1.3 – Timeline of the project

A Gantt Chart has been created for this FYP. This chart includes the tasks needed to be completed within the whole FYP period (34 Weeks excluding winter break and recess week). The purple blocks represent the planned schedule for the project. The green blocks represent the actual time taken to complete the task.





Chapter 2 – Literature Review

In this chapter, we would explore the mobile development frameworks and tools used in the development of this project. Additionally, this chapter would also explain why these technologies were used for the development of this project.

2.1 – Researching on known mobile development frameworks

In this subchapter, we would like to discover the various frameworks used in developing mobile applications.

2.1.1 – React Native

In the market, there are various software available for the development of mobile applications. Firstly, React Native. This is something new and the most recommended for the development of Mobile Applications. It is created by Facebook also known as Meta. It can be used to develop both Android & iOS. It uses a single code base for both Android and iOS applications [1]. The benefits of using React Native are:

- Code Re-usability
- Compatible with third-party plugins
- Re-usable components for optimal performance
- Provides hot deployment features
- Ease of Maintenance

2.1.2 – Flutter

This is another popular mobile development framework developed by Google. It is a UI tool kit to build native applications for mobile, desktop and web applications. It is a cross-platform mobile app development framework that works with one codebase that works with both Android and iOS [1]. Benefits of using Flutter are:

- It Provides Full Native Performance
- Flexible UI
- Provides Strong Widget Supports

- Offers built-in Designs
- Fast App Development

2.1.3 Mobile Angular UI

It uses an open-source mobile application development framework that merges the implementation of the Bootstrap and Angular frameworks. It offers various UI components like as overlays, switches, sidebars, navbars, scrollable areas to provide robust and attractive user interfaces for the applications created [1]. The benefits of Mobile Angular UI are:

- Responsive Framework
- Offers Interactive Components that is built-in
- Lightweight means it's easy to use
- No jQuery dependencies
- Easy conversion from web or desktop to mobile

2.1.4 – Ionic

Ionic developed in 2013 is an open-source framework that allows a cross-platform building for mobile apps using web technologies like HTML, CSS & JavaScript. It can work on Android, iOS and Windows platforms [1]. The benefits of Ionic are:

- Fast App Development
- Built-in UI components available
- Platform Independent
- Based on AngularJS

2.1.5 – Xamarin

Xamarin is an open-source frameworks used to develop mobile applications. It is based on .Net and allows one to build native applications for Android, iOS, and Windows platforms [1]. The benefits of Xamarin are:

- Huge Community of around 1.4 million developers
- Native API Access & UI Support
- Easier API Integration

- Target All Platforms
- Cost-Effective & Faster Development Process

2.1.6 – Adobe PhoneGap

It is purchased by Adobe Systems in 2011, and it is an open-source framework that allows you to build cross-platform applications using languages such as HTML5, CSS3, and JavaScript [1]. The benefits of Adobe PhoneGap are:

- Code Re-usability
- No Hardware Restrictions
- Plugin Library & Third-Party tools
- Robust Support System
- Compatibility with all the platforms

2.1.7 - Appcelerator Titanium

It is developed by Appcelerator Inc and is an open-source development framework works on all mobile platforms. It is based on a Javascript-based SDK with over 5000 APIs for Windows, HTML5, iOS, and Android that further allows the reusability of Javascript code up to 90% in case you're targeting multiple platforms for application development [1]. The benefits of Appcelerator Titanium are:

- Various Features for API Management
- Compatible with multiple platforms
- Easier API Integration
- Better Code Reusability
- Faster Development Process

2.1.8 - Framework 7

It is an open-source development framework that allows the development of mobile, desktop, or web applications with a native perspective. It supports HTML, CSS, & JavaScript and

offers the additional support of Vue.js and React components for app development [1]. The benefits of Framework 7 are:

- Easier Customization
- Provides various UI widgets and elements
- Paired with additional tools like Electron & NW.js
- Rich Ecosystem
- Ease of Maintenance

2.1.9 - JQuery Mobile

It is a cross-platform development framework that is used to build mobile and web applications for various devices. It is a HTML5-based development framework, and it comes with various plugins such as Image Slider, Content-Slider, Pop-Up Boxes, etc. that help to create a feature-rich and interactive design [1]. The benefits of JQuery Mobile are:

- Responsive Framework
- Supports multiple platforms & devices
- Lightweight Size
- Various Add-on Plugins
- Powerful Theming Framework

2.1.10 – NativeScript

It is an open-source framework that is used to build native mobile applications using Angular, Vue.js, TypeScript, or JavaScript. It allows one to create both Android and IOS apps from a single codebase [1]. The benefits of NativeScript are:
Cross-Platform Development

- Direct access to Android & iOS APIs
- Native Performance
- Re-usability of Code
- Strong Backend Support

2.1.11 – Android Software Development Kit (SDK)

It is a collection of libraries and Software Development tools that are essential for Developing Android Applications [2]. The benefits of Android Software Development Kit (SDK) are:

- Allows for Live Edit
- Intelligent Code Editor
- Debugging Tools – LogCat allows the user to see real-time device logs and app logs
- Profiler – Gives live information about CPU, memory, network
- Visual Layout editor
- Resource Manager – Vector Asset Studio(For Icons)

2.2 – Discovering the various AI API models

In this subchapter, we discover the various AI API models that exists.

2.2.1 – DALL E

It is used to create images based on a verbal description. It can create images from scratch and edit ones that exists [3].

2.2.2 – GPT

GPT – 4 and newer are increasingly multimodal. This means that it can process and understand both text and images. It allows for more sophisticated image recognition tasks [3].

2.2.3 – Gemini

Gemini is multimodal that allows for text, image, audio and video recognition. It can generate descriptions for images, detect objects and bounding boxes, answering questions about an image’s content [4].

2.3 – Cloud-Based Database for PhytoLens

There is a need for a cloud-based database for PhytoLens. Here are the common databases used for mobile application development.

2.3.1 – Firebase

It is a database that is owned by Google. It has features like authentication, real-time DB, cloud storage, cloud messaging and hosting services. Using firebase is easy as it has an easy integration into both Android and iOS [5].

2.3.2 – SQLite

It is an embedded database that is widely used in mobile application development. It is well known for being simple, reliable and compatible with both Android and iOS [5].

2.3.3 – Realm

This is a mobile database that replaces SQLite and core data. Realm provides a real-time data synchronization, mapping objects, and a simple API that suits mobile development [5].

2.3.4 – Couchbase Mobile

This is a NoSQL database. It provides features like data synchronization, offline data support and full text search [5].

2.3.5 – Amazon DynamoDB

This DB is owned by Amazon, and it is too a NoSQL database. It offers a high availability, scalability and performance. It also supports essential features like data encryption, backup and restore [5].

2.3.6 – MongoDB Realm

This is a mobile database platform that offers a combination of Realm's local storage and MongoDB's scalable backend. It provides features such as user authentication, synchronization, and serverless functions [5].

2.3.7 - PostgreSQL

This is a powerful RDBMS and is known for its robustness, extensibility and adherence to SQL standards. This is primarily used for server-sided deployment [5].

2.4 – Selecting the tools for development

In this subchapter, we would be choosing and explain why we chose the tools that we are going to use for the development of PhytoLens.

2.4.1 – Choosing the Framework

For the Framework, we chose Android Software Development Kit (SDK) as it is the easiest to use and implement. On top of that, time is of essence as developing an application in two semesters on top of doing finals requires a lot of time. Prior experience in Android SDK also allows for a quicker development of PhytoLens.

2.4.2 – Choosing of the Database

For the database, we chose Firebase as it is the easiest way to integrate the database into the application. On top of that, Android Studio has a built-in feature to integrate Firebase smoothly.

2.4.3 – Choosing of the AI API model

For the AI API model, we chose Gemini as it is free to use and the amount of images being sent to the prompt is limitless. On top of that, the details being produced after identifying the image, it identifies the images correctly and gave good descriptions.

Chapter 3 – Development and Implementation

This chapter will show the development and implementation of PhytoLens. Each of the features implemented so far will be shown in detail in the sub-chapters starting with the front-end development followed by the back-end development.

3.1 – Front-End Development

3.1.1 – PhytoLens Landing

In the main screen, the user should be able to see the image in (Figure 1). At this point of time, only the capture and the profile screens have been implemented. As for the other buttons and the search feature in the main screen, it has not been implemented yet.



Figure 1: Landing Page of PhytoLens

At the bottom of the screen (the navigation bar), there are three buttons. The Home button that leads to the main screen of PhytoLens, the Capture button that goes to the capture screen and opens the camera within the app and the Profile button that leads to the profile screen. (Figure 2) will show the graphics of the button redirections.



Figure 2: PhytoLens landing screen + Navigation button redirections

The home page is designed in such a way that the elements are constrained within a constraint layout format where it allows for the elements within the activity be tied to one another within a fixed width or height. This is better than a traditional linear layout as a constraint layout is flexible and depends on the constraints applied to its parent or siblings. It is adaptive to the width and height of the application. So, for this case the search bar is constricted to the top of the parent which is the background. The search button is constrained to the search bar and so on as the element goes down. For the navigation bar, the Home button is coded such that it will direct to the home page upon tapping by using the code:

```
Button homeButton = findViewById(R.id.bottomHome);

    homeButton.setOnClickListener(new View.OnClickListener() {

      @Override

      public void onClick(View v) {

        Intent intent = new Intent(MainActivity.this, MainActivity.class);

        startActivity(intent);

        finish();

      }

    });
```

The first line "**Button homeButton = findViewById(R.id.bottomHome);**" is meant to find the id set in the activity layout file to put functions to the button. The line after that then sets a function of onclick to create a new view to go to a page set within the onClick method that comes after that. The intent with a capital I is a built-in function in Android Studio. The small i intent is the variable name created within the method. Within the initiation of the Intent code, the activity.this stands for where the page is at currently and the class is where it would direct to upon clicking the button. After initialisation, the startActivity(intent) ensures that the page is being launched upon clicking the button. This is the same for all the buttons coded.

3.1.2 – Capture screen

In this screen, there are 3 buttons at the bottom of the screen and a cross button at the top left of the screen. In the centre of the screen the user is given the option to choose to identify insects or plants (Figure 3) before the camera is initialised. After choosing what to identify, the camera would appear and the identifying process can start. This can be seen in (Figure 4).



Figure 3: Choosing to identify plants or insects



Figure 4: Capture screen

Figure 4 was made using the androidx camera lifecycle module. On top of that, the AndroidManifest.xml had to be modified by adding the code to allow for the device's camera to be used within the application:

"<uses-permission android:name="android.permission.CAMERA" />"

The starting of camera is coded within a startCamera method within the camera activity java code as seen in (Figure 4).

```
public class CameraActivity extends BaseActivity {
    2 usages
    private void startCamera() {
        cameraProviderFuture = ProcessCameraProvider.getInstance( context: this);

        cameraProviderFuture.addListener(() -> {
            try {
                ProcessCameraProvider cameraProvider = cameraProviderFuture.get();

                int targetAspectRatio = androidx.camera.core.AspectRatio.RATIO_4_3;

                Preview preview = new Preview.Builder()
                        .setTargetAspectRatio(targetAspectRatio) // Apply ratio
                        .build();
                preview.setSurfaceProvider(cameraPreviewView.getSurfaceProvider());

                ImageCapture = new ImageCapture.Builder()
                        .setCaptureMode(ImageCapture.CAPTURE_MODE_MINIMIZE_LATENCY)
                        .setFlashMode(ImageCapture.FLASH_MODE_AUTO)
                        .setTargetAspectRatio(targetAspectRatio) // Apply SAME ratio
                        .setTargetRotation(cameraPreviewView.getDisplay().getRotation())
                        .build();

                CameraSelector cameraSelector = CameraSelector.DEFAULT_BACK_CAMERA;
                cameraProvider.unbindAll();

                androidx.camera.core.Camera camera = cameraProvider.bindToLifecycle(
                        lifecycleOwner: this,
                        cameraSelector,
                        preview,
                        imageCapture // Bind individual use cases (simpler)
                );
                cameraControl = camera.getCameraControl();

            } catch (ExecutionException | InterruptedException e) {
                Log.e(TAG, msg: "Error starting camera: " + e.getMessage());
                Toast.makeText( context: this, text: "Error starting camera: " + e.getMessage(), Toast.LENGTH_LONG).show();
            }
        }, ContextCompat.getMainExecutor( context: this));
```

Figure 5: startCamera method to use the camera function

The first button at the bottom of the screen on the left allows the user to go to their phone's photo gallery to choose an image from there. Upon clicking, the phone's gallery should pop up and the images from the gallery should be populated there. This is seen in (Figure 6)



Figure 6: Image gallery (Cropped out the bottom to only show relevant items(plants))

The second button in the centre allows the user to snap an image using the phone's built-in camera. This is seen in (Figure 7)



Figure 7:Snapped image of a plant

This is done by using the ImageCapture function called takePicture which is a function in androidx camera core which is a part of the CameraX core. This allows for the image to be captured. For the image to be displayed on the screen, we used something called Bitmap to display the image.

The last button in the navigation bar shows tips on how to take an image for a more accurate analysis of the plant or animal. It is a pop up that will appear and would disappear when the user taps anywhere on the screen. This can be seen in (Figure 8)

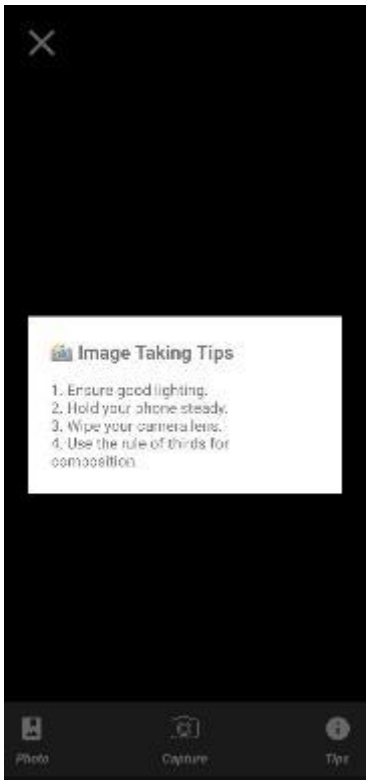


Figure 8: Tips button

As for the last button on this screen, the X button on the top left, it would redirect the user back to the main screen on click. So, the user would see (Figure 1) upon clicking this button.

3.1.3 – Analyse image screen

Within this page, users are given the choice of using the image they took and use it for analysis. They can either tap on the Analyse button or Retake button. If the user taps on the Retake button, they would be redirected to the capture screen (Figure 3). If the user taps on the Analyse button, the image would be analysed, and the Analysing Image text would pop up in the middle of the screen as seen in (Figure 9).



Figure 9: Analysing of image

This page was developed by sending the Bitmap of the image taken earlier to the AI API by using this prompt for the AI:

"Identify this " + selectedMode + ". " + "Return ONLY JSON: {'is_detected': bool, 'name': string, 'poison_status': string}. " + "CRITICAL: The 'name' field must ONLY contain the common English name (e.g., 'Rose'). " + "Do NOT put the scientific name in brackets beside it.";

This ensures that the correct details are displayed and making sure that only plants are being identified based on the image taken and provided to the AI.

3.1.4 – Results of analysing image

After analysing the image, the user would be directed to the Results screen where the Only the image taken by the user, name of plant/insect and the poison status would appear as these are the essential information. This can be seen in (Figure 9).



Figure 10: Results after analysing image

For more detailed information, the user can tap on the “Want More Information” button to get in-depth information of the analysed plant/insect. The details shown when tapping on the “Want More Information” would be things like the Scientific Name, Fun Facts, Recipes and Variations. This can be seen in Figure 11

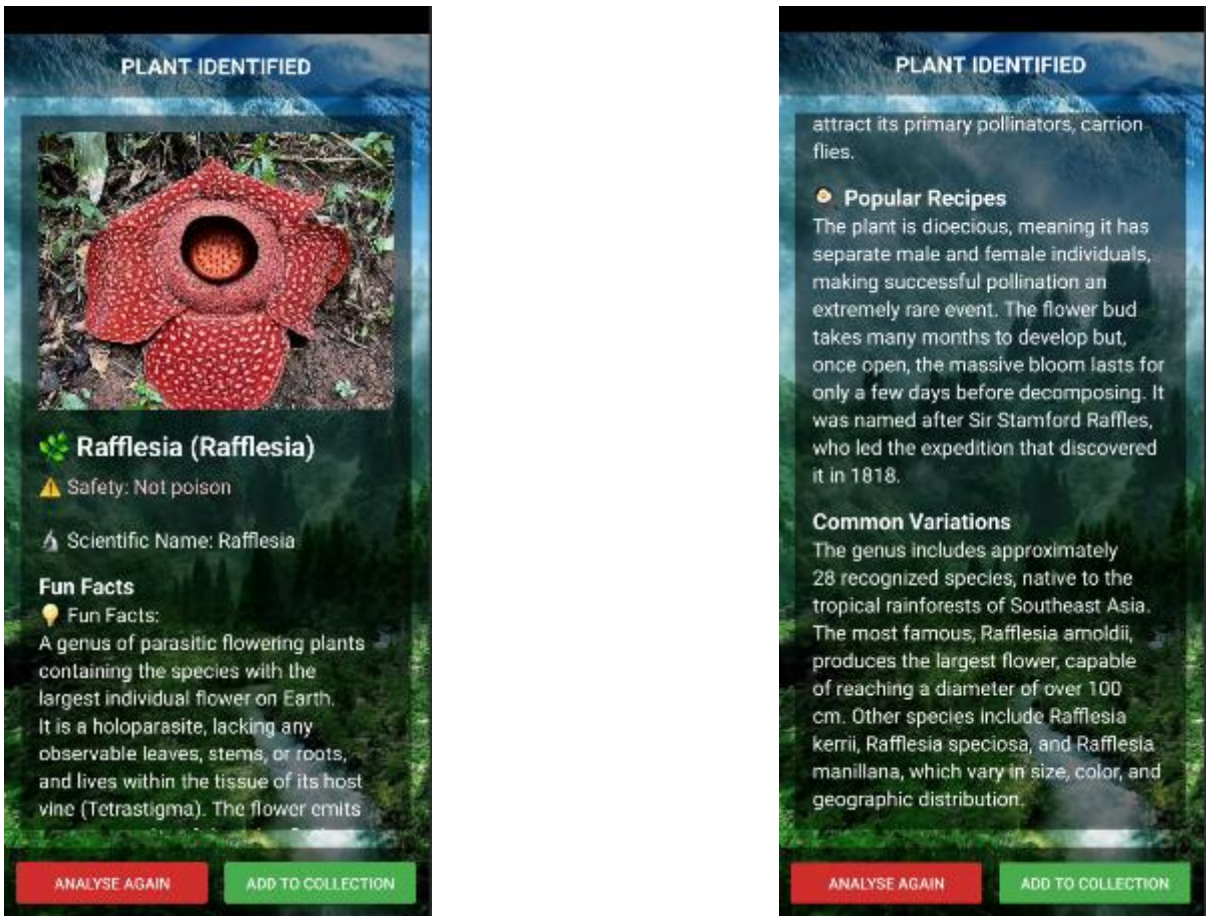


Figure 11: Deep Dive analysis

At the bottom, there would be 2 items. The first is button, Analyse again at the bottom would direct the user to the Capture Screen (Figure 3) on tap. The Add to Collection button on the right would add the details generated here together with the image taken to the device’s storage and firebase.

3.1.5 – Analysis History

Upon clicking the Add to Collection button, the user would be redirected to this screen. This screen is made up of a scroll view where all the previous analysis of plants would appear here from the time the application was installed. The details here would be removed upon the deletion of the application as it is stored on the application's local storage. The look of the screen can be seen in (Figure 10)

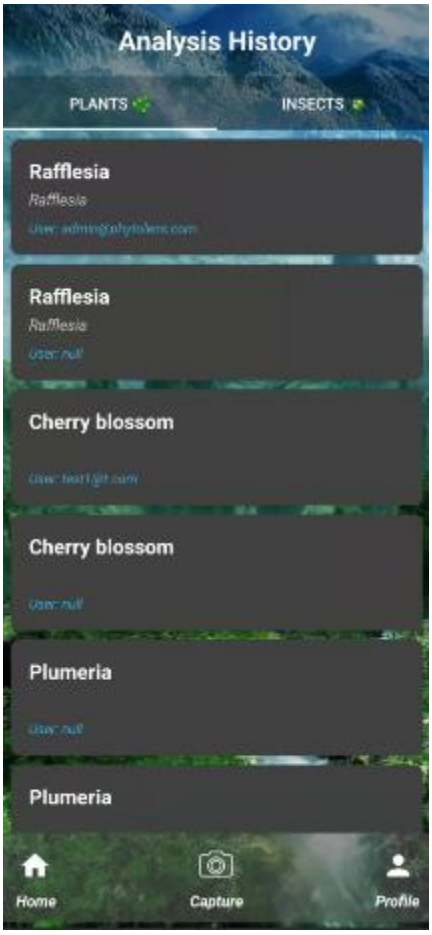


Figure 12: Analysis History screen (plants)

Here users can scroll through the list of items they had analysed before, and they can tap into the specific item. Upon tapping in, they can see the details of the plant they analysed in (Figure 10). At the bottom of the screen is the same navigation bar explained in Chapter 3.1.1. This page too has a filter to filter between plants and insects so that it is more user friendly. Figure 12 shows the plants filter and Figure 13 shows the insect's filter.

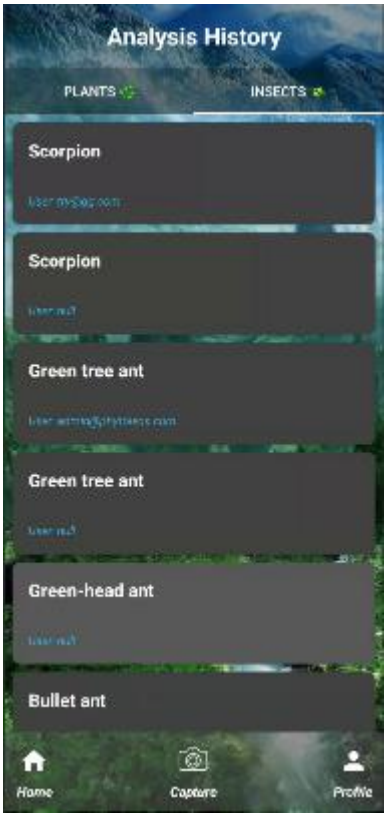


Figure 13: Analysis History screen (insects)

This page was developed using two layout files. The first is the main that uses a constraint layout to align every element. The list items are done using a RecyclerView within a constraint layout. The values like the layman's name and the scientific name are designed in a separate layout file. This layout file is then called within the RecyclerView in the form of tool.

3.1.6 – History Details

In this subchapter, there are two options. The details would display basic information only if the user did not select the “Want More Information” button earlier as seen in Figure 14, but the button is now replaced with “Get Deep Dive Info”. However, if the user had opted to tap on the “Want More Information” button earlier, then they would see the full details of the analysed plant/insect as seen in Figure 15.

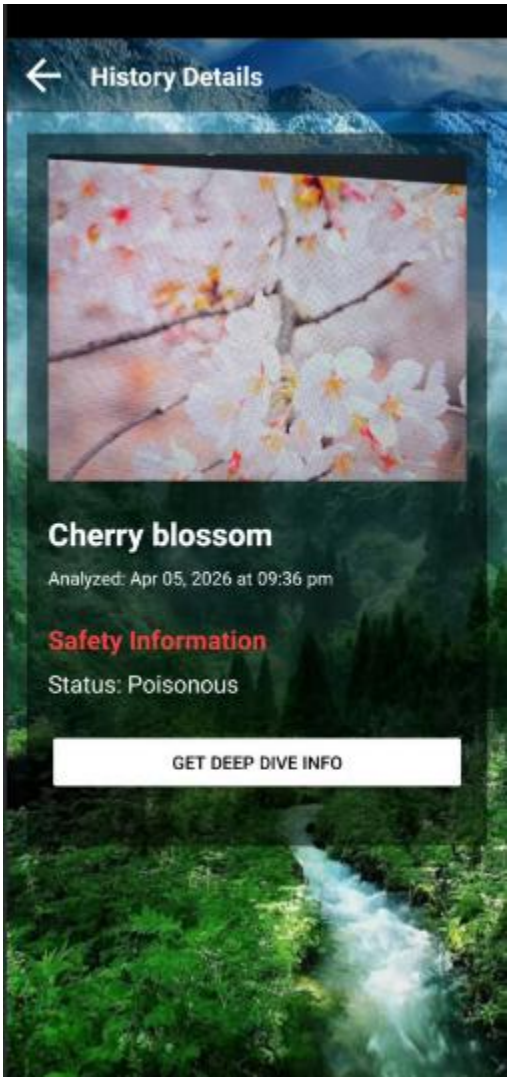


Figure 14: History without more info



Figure 15: History with more info

This page is developed by saving the information generated from the previous analysis and then calling the saved data here. Each analysis is saved into the Firebase database so that it

can have its own unique ID. By having a unique ID, it enables the data to be saved systematically, and no detail of that analysis would be lost since it is tied to the ID.

The back button on the top left goes back to the Analysis History screen (Figure 13).

3.1.7 – Profile screen

In this screen, users can see the images that they had analysed in the past since the installation of the application. If the user is logged in, they can see everything that they had analysed on the device since they installed it plus the analysis they did before when they are logged in in a different device. On top of seeing the analysis history, they can also see the login and create new account buttons if they are not logged in. (Figure 16)

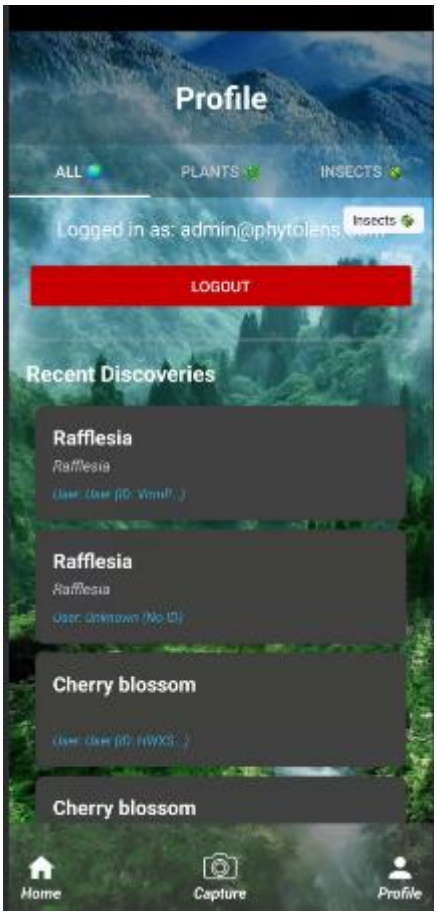


Figure 16: Profile Page

In the profile page, users can filter to see either All their previous analysis or plants only or insects only. This would make it more user friendly since users are given a choice. Figure 16 shows everything. Figure 17 shows plants only and Figure 18 shows insects only.

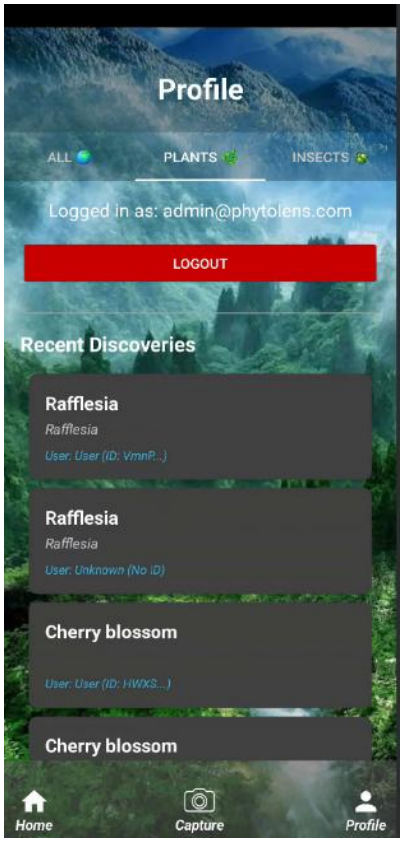


Figure 17: Profile Page Plants Filter

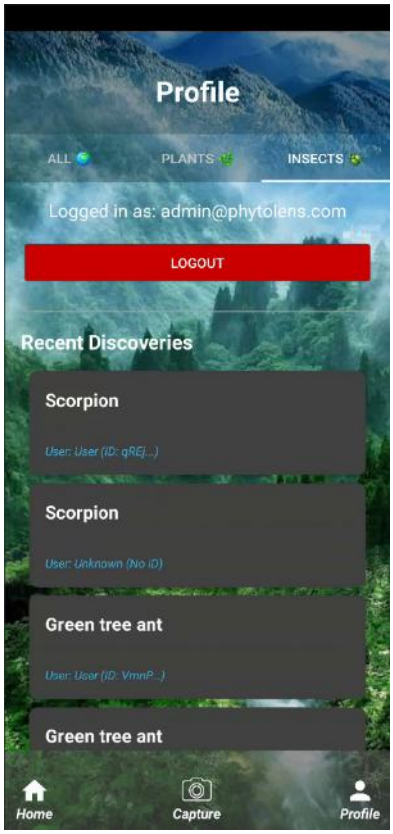


Figure 18: Profile Page Insects Filter

When the user taps on the Login Button, they would see the login form where the user needs to input the email they used to sign up and their password. If the user taps on Create New Account instead, they would be directed to the new account creation form where they can see the fields that asks for name, contact number, email, password and confirm password (Figure 19).

The previous analysis can be found here too. The way it was coded is like that of Chapter 3.1.5. The only difference is that it has the login and create new account buttons.

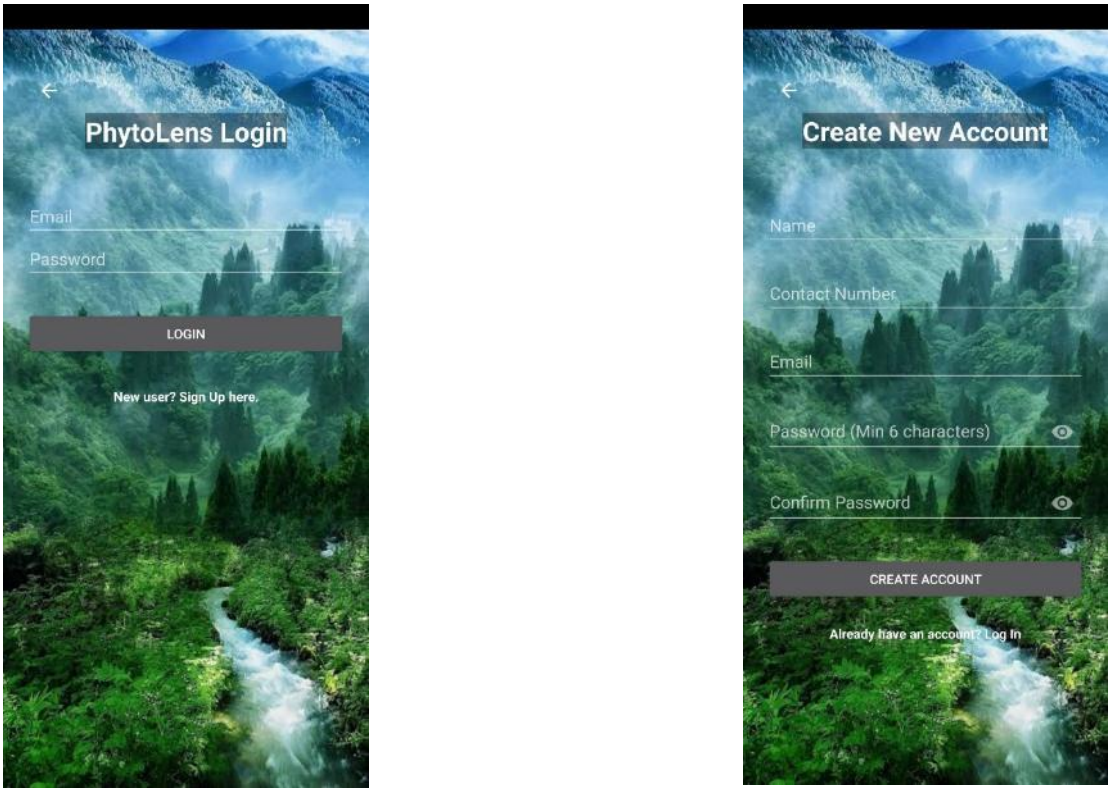


Figure 19: Login and Create New Account screens with their respective forms

In the Login screen, if the user taps on the “New user? Sign Up here.” text, they would be directed to the Create New Account screen. Same goes if the user taps on the “Already have an account? Log in” text on the Create New Account screen, the user would be directed to the Login screen.

These screens are made in conjunction with Firebase where the Authentication and Firestore Database modules from Firebase are used. The authentication module is where the email and password are stored. It will check against existing users to ensure that there are no duplicate users when creating and do a password check when logging in to ensure that the password is correct. On top of storing the email and password in the authentication when creating a new account, the contact details and username would be stored in the Firestore Database. This is done by mapping the fields to specific variables in the database where the FirebaseFirestore function is called after. This is seen in the code below:

Map<String, Object> userProfile = new HashMap<>();

```
userProfile.put("email", user.getEmail());
userProfile.put("name", name);
userProfile.put("contact", contact);
userProfile.put("createdAt", FieldValue.serverTimestamp());
// Save to 'users' collection with UID as the document ID
FirebaseFirestore.getInstance()
        .collection("users")
        .document(user.getUid())
        .set(userProfile)
```

3.1.8 – Admin login

If the admin is logged in, they can see all the analysis done by all the users in their profile page. It also shows the user that analysed the image together with the time and date it was analysed. At this point UID of the account can be retrieved. (Figure 20). The look here is different from the profile screen of a normal user (Figure 21).

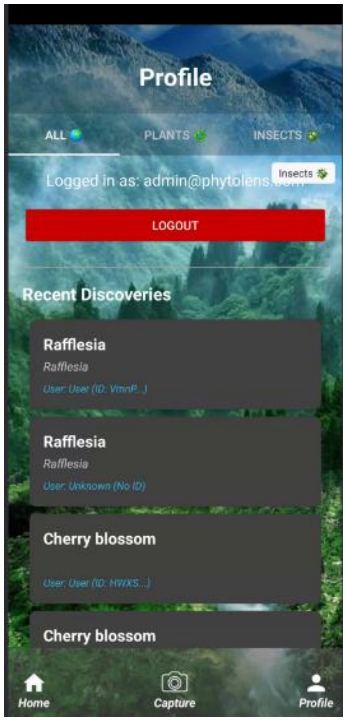


Figure 20: Admin account



Figure 21: Normal User Account

The admin determination is set in code by getting the UID from Firebase Authentication. There are checks in the code that checks for the user logged in based on the UID. If the UID is that of the admin then they will see (Figure 20). If the UID is not that of the admin then they will see (Figure 21). If the user is not logged in, then they will see (Figure 22).



Figure 22: Guest User Profile

3.1.9 – Search

This subchapter, we would explore the search function of the application. We first tap on the search bar in the hope page and then type in the plant that we want to search up more as seen in Figure 23.



Figure 23: Search function in mainpage

In the search bar, we can enter the things that we want to search in this case we try apple. Then after keying in the plant, we hit on search then it would bring to the next page where the search would begin and then it would display out the values as seen in Figure 24 where details are shown together with the Wikipedia link.



Figure 24: Search result

3.1.10 – Types of plants

In this subchapter we would explore a feature where a user can click on types of plants in the main screen as seen in Figure 1. In Figure 25 shows the moment the button is clicked.

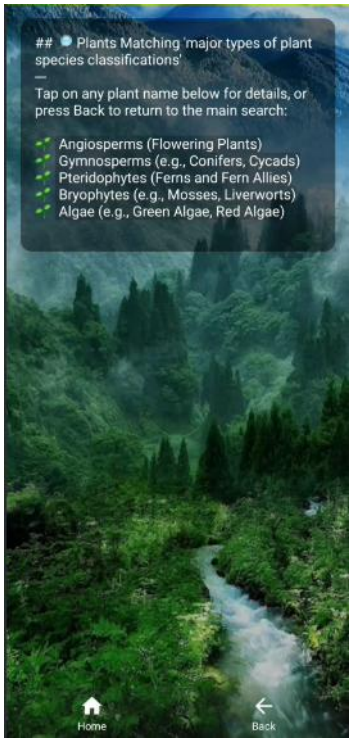


Figure 25: Types of Plants

Here users can tap on any of the categories and find out more about them. The details would be the same of that of Figure 24. The home button and back button leads to Figure 1.

3.1.11 - Photosynthesis

In this subchapter, we would talk about another feature which is called Photosynthesis. It is a button where users can click and then find out more about photosynthesis. So, from the mainpage (Figure 1), the user would tap on the photosynthesis button and then would be redirected to a page that have details about photosynthesis as a whole down to the chemical compounds as seen in Figure 26.

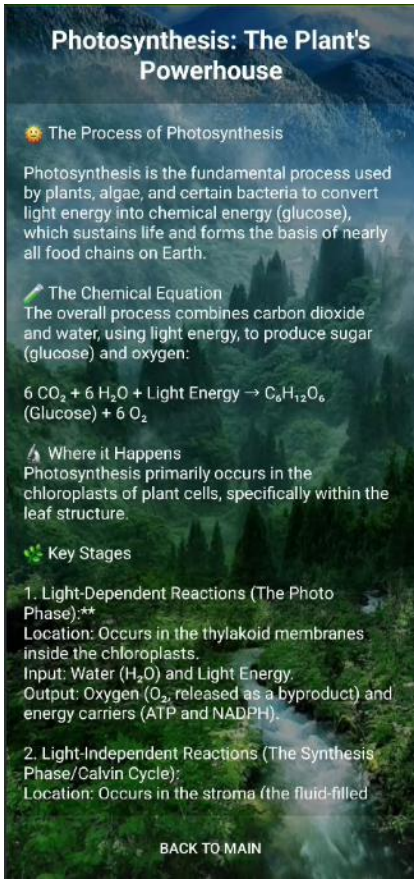


Figure 26: Photosynthesis

Here users get to read and understand the process of the plant's most important process and get to know how this came about. On the page, a "Back to Main" button is present at the bottom that when on clicked would bring the user back to the main page.

3.1.12 – Fun Facts

Here we would discuss on the fun facts about plants. Users can tap on the Fun Facts button in the main page to take them to the Fun Facts page. Here is where they can find things like the largest tree and the oldest trees as seen in Figure 27.



Figure 27: Fun Facts

3.1.13 – Articles

In this subchapter, users can choose to read articles that are related to plants like how to start botany at home, deforestation and more by clicking the more articles button we can see the various articles in Figure 28.

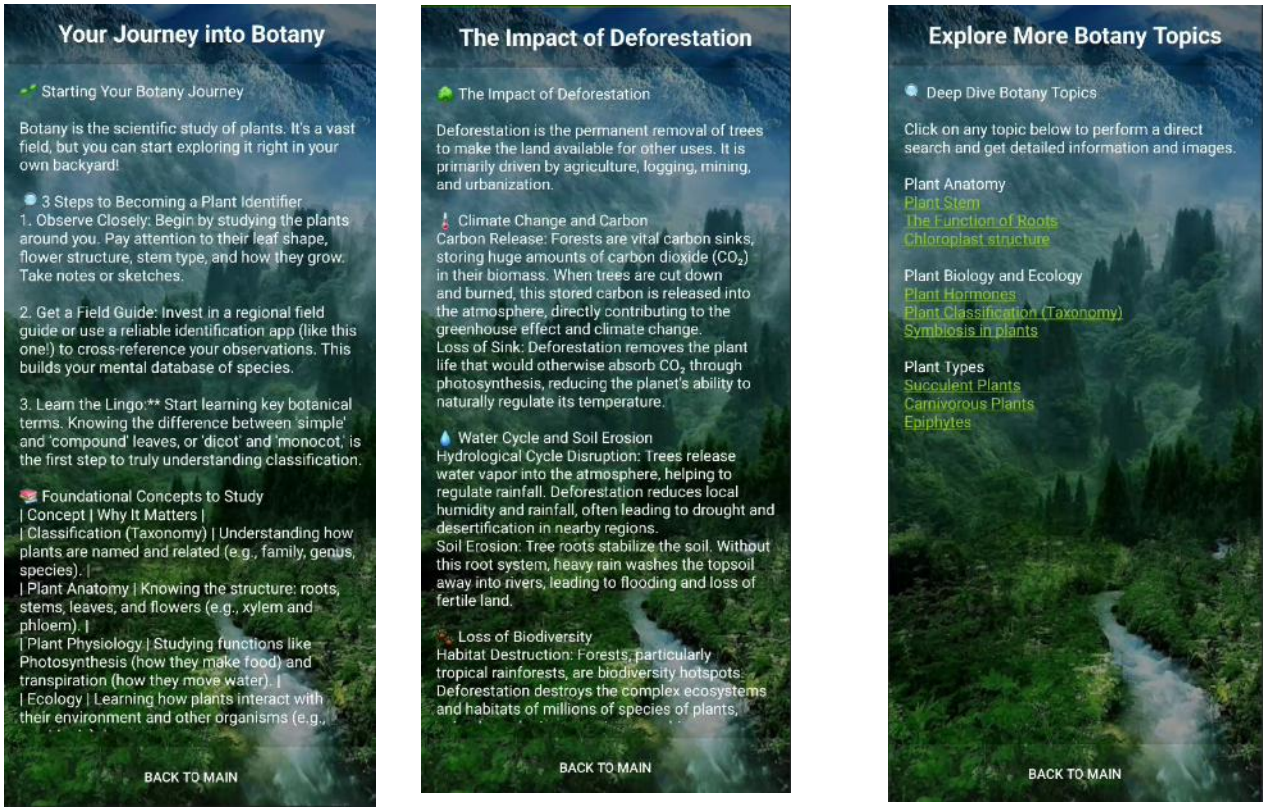


Figure 28: Articles

3.1.14 – Support / FAQ

Here users can seek support or read the frequently asked questions (FAQ). Users can access this support by tapping on the Support / FAQ button at the bottom of the main page as seen in Figure 1. Upon redirecting to the Support / FAQ page, there would be various asked questions and answers for users. Also, there is an additional support link where users can tap on which would direct them to the email app to mail the support team. This can be seen in Figure 29.

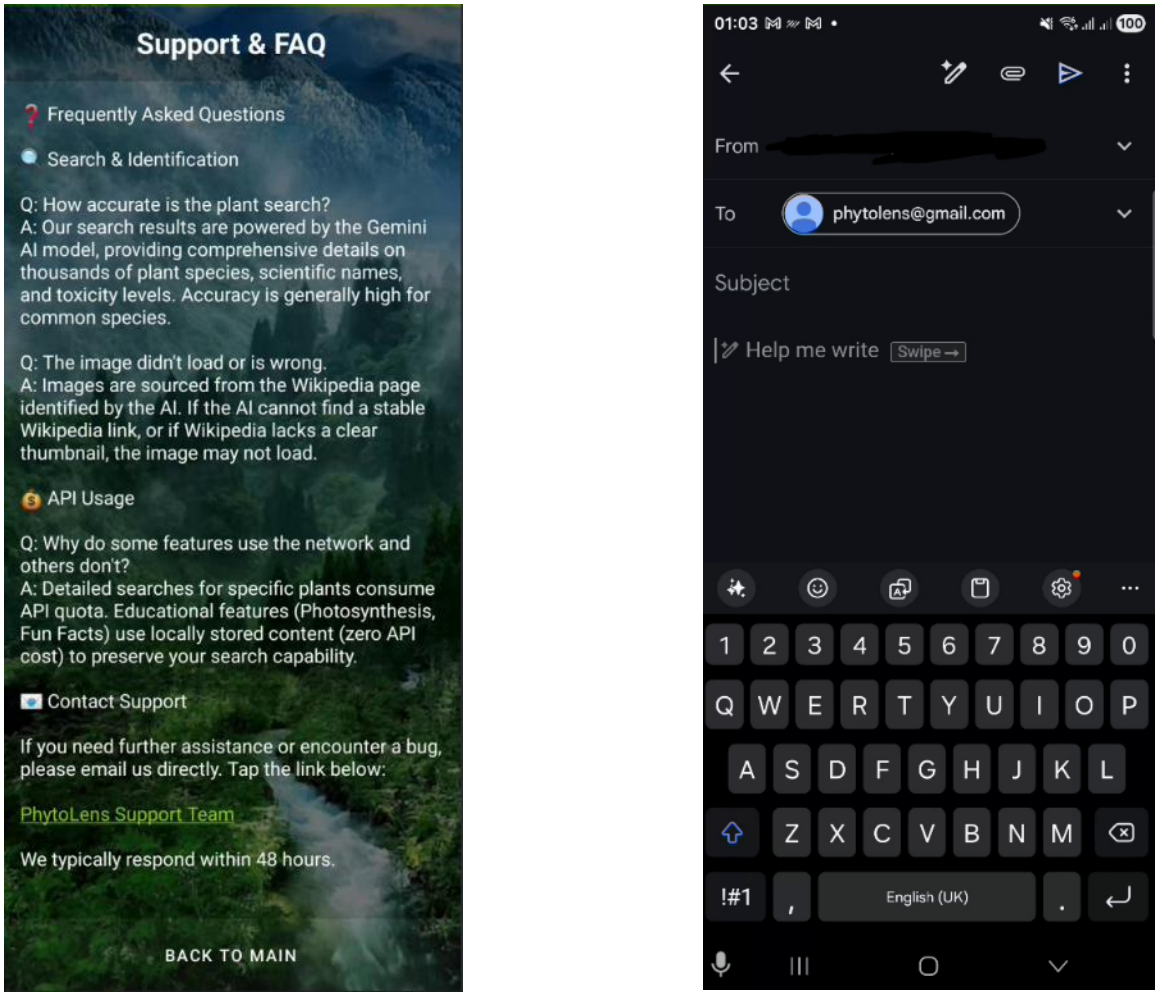


Figure 29: Support/FAQ and email

3.1.15 – Forums

Here users can post information to the forums and comment on them. This acts as a forum that filters based on the discussion topics. Admins and moderators can delete posts and comments that are inappropriate. The users can tap on the Forums button at the bottom of the main page, and they would be redirected to the forum community topics page where they can choose the topic that they want to ask questions or comment. Figure 30, 31, 32, 33, 34 shows the whole forum from selecting the topic, creating of post and commenting on post.

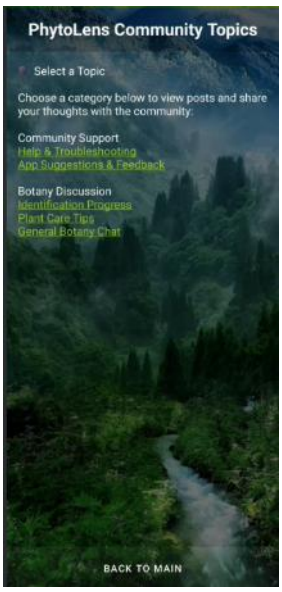


Figure 31: Forum Page



Figure 30: Forum Topic

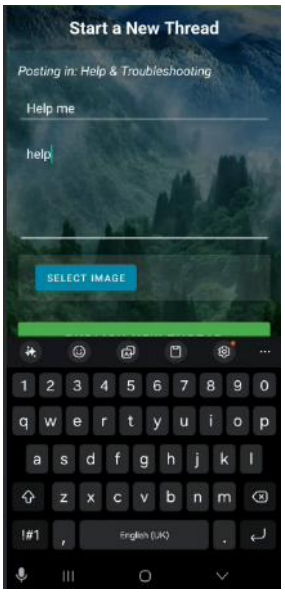


Figure 32: Start New Thread



Figure 33: Posting Comments

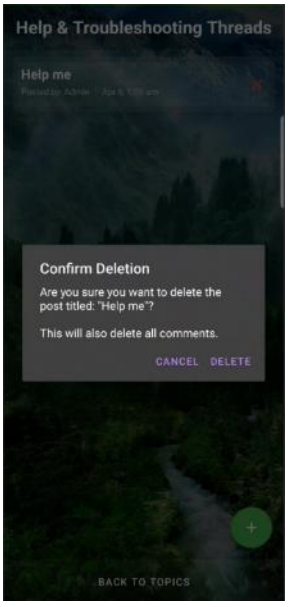


Figure 34: Admin Delete Thread

3.2 – Back-End Development

In this chapter, we will discuss about the implementation of Firebase for the back-end development.

3.2.1 – Creation of Project

After creating the PhytoLens project in Firebase, I am directed to the Project Overview page (Figure 35)

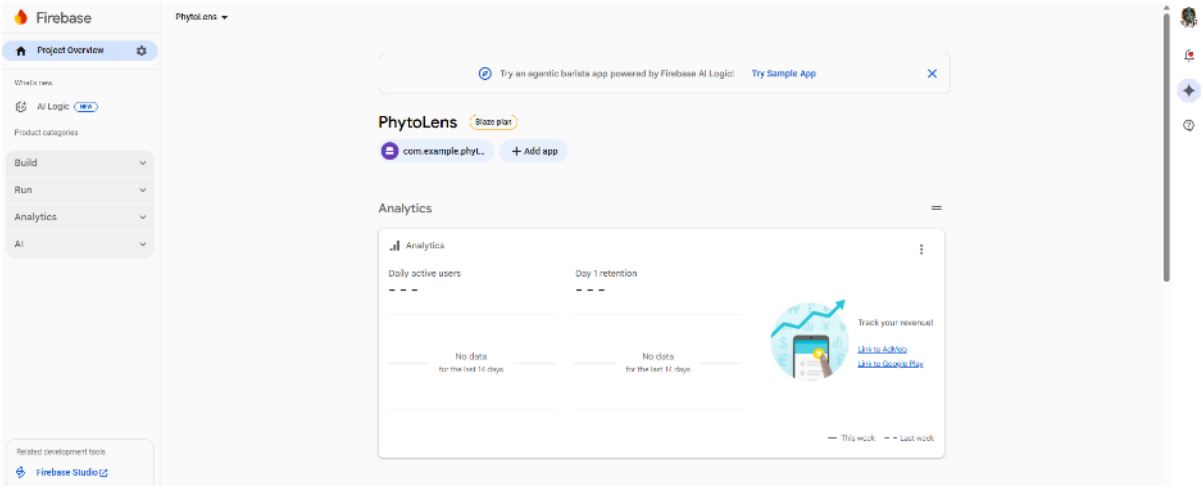


Figure 35: Firebase PhytoLens Project Overview

In this page, I would then go to the Build option on the left side of the screen and select 3 things. Authentication, Firestore Database and Storage (Figure 36).

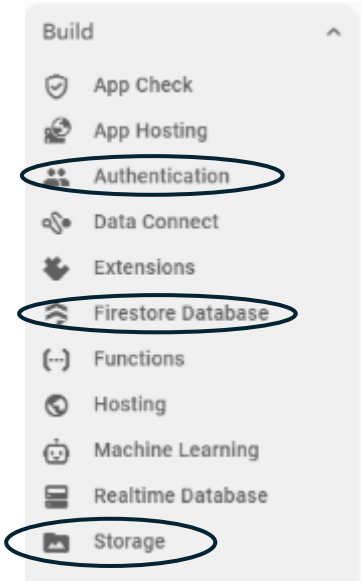


Figure 36: Choosing of Firebase functions for PhytoLens

3.2.2 – Setting up the Authentication

Within the authentication function in Firebase, I had enabled Email/Password as the Sign-in method since that is what we want, and the Users screen shows all the accounts with the UIDs that I have created so far. Only the admin has been created in the Firebase Console.(Figure 37)

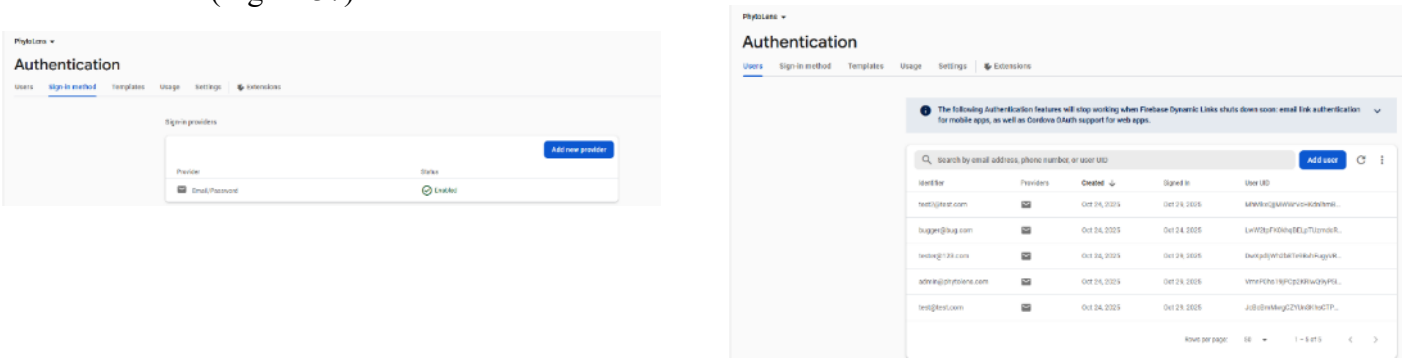


Figure 37: Authentication setup

3.2.3 – Firestore Database setup

Here is where I had already set up to store the APIKey for the AI, InsectHistory, PlantHistory, Forum Messages and users that stores the user's details when they sign up (Figure 38).

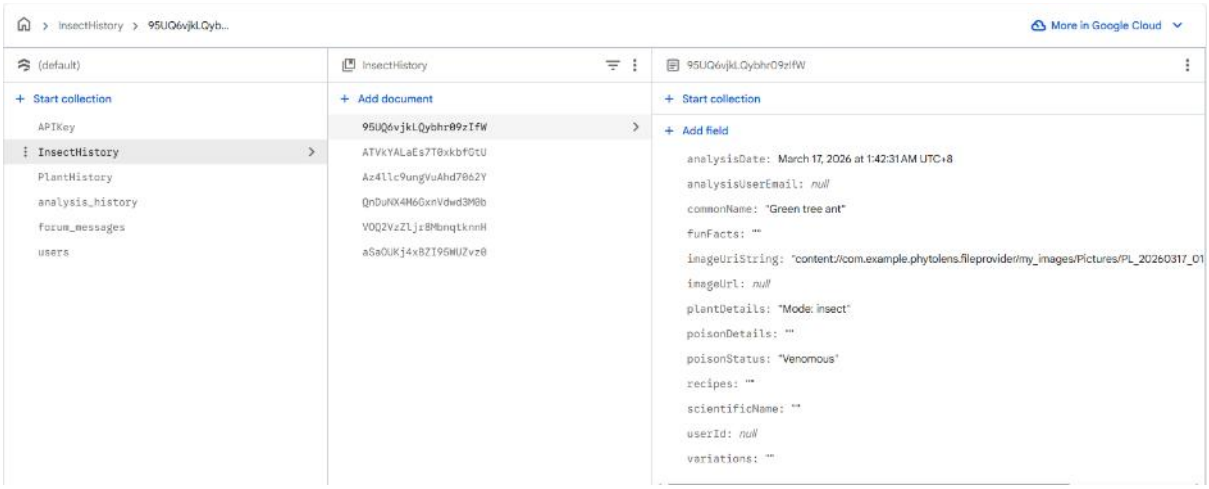


Figure 38: Firestore Database

3.2.4 – Firebase Storage

In this chapter, this is the Firebase Storage where all the analysis_images are stored. The images stored are the images taken by the users.(Figure 39)

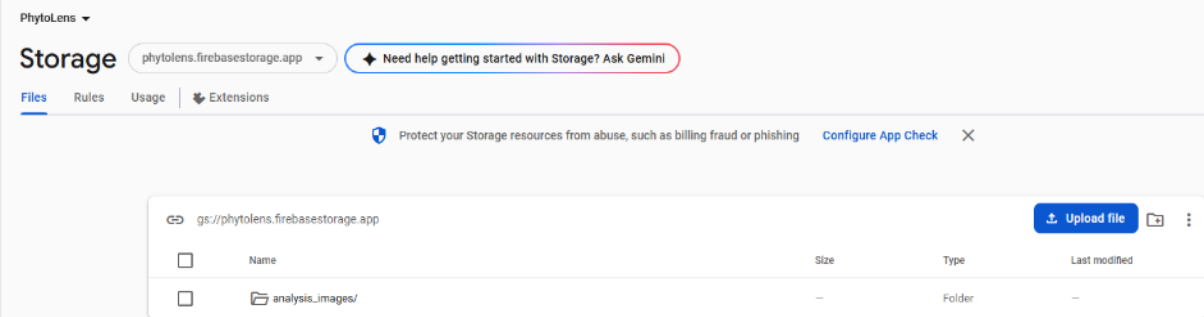


Figure 39: Firebase Storage analysis_images

Inside the analysis_images folder, I have separated the images taken by guests or logged in users. The logged in user's folders are based on the UID of the user's account (Figure 40).

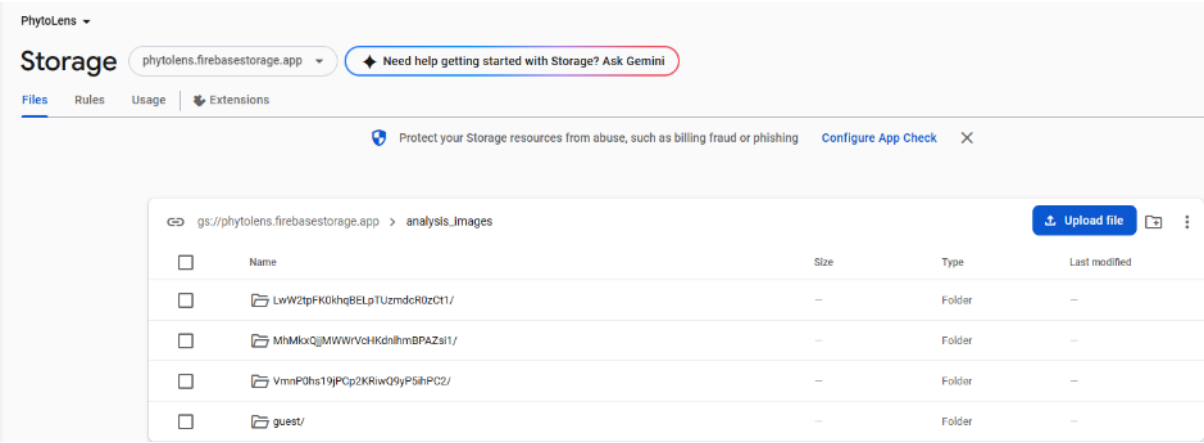


Figure 40: Segregation between registered users and guests' images

3.2.5 – Comment Images

This chapter is where the images that users upload when commenting on forum topics is stored. As seen in Figure 41, this is the folder and then in Figure 42 is the folders of the images. The folders would be named based on the user's id.

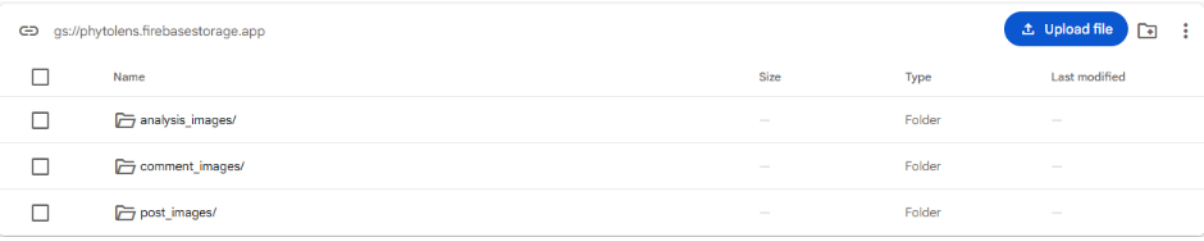


Figure 41: Storage

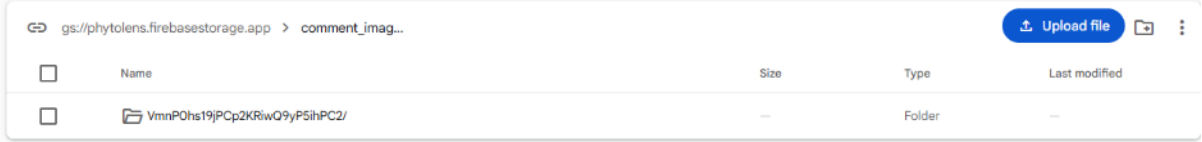


Figure 42: Inside Comment Images folder

3.1.6 – Thread Post Images

Here we can find the images uploaded by users when they created a thread. Same as the comment images, it would be filtered based on the user ids in Figure 43.

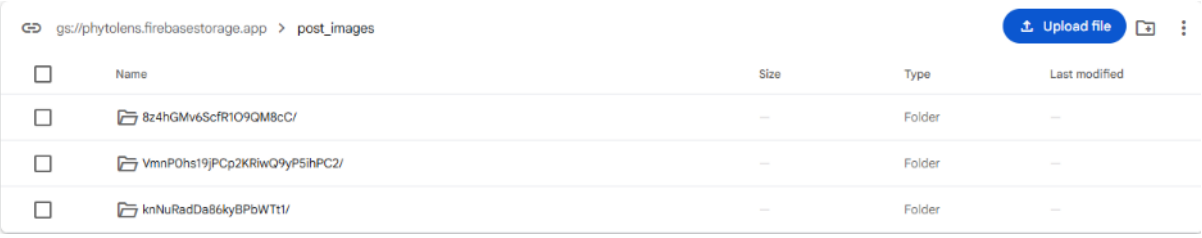


Figure 43: Inside Post Images folder

Chapter 4 – Testing and Results

In this chapter, we would look at the various test cases and results for PhytoLens.

4.1 – Testcase 1: Dynamic Identification

Scenario:

We need to test PhytoLens can correctly identify based on the selected mode. In this scenario we are going to use the plant identification mode to try and analyse insects.

Expected Results:

Should not return the name of the insect and should prompt up an error saying that nothing was detected. No results should be returned in the analysis screen.

Steps:

Here are the steps taken to test the scenario.

Step 1:

Tap on Capture ✓

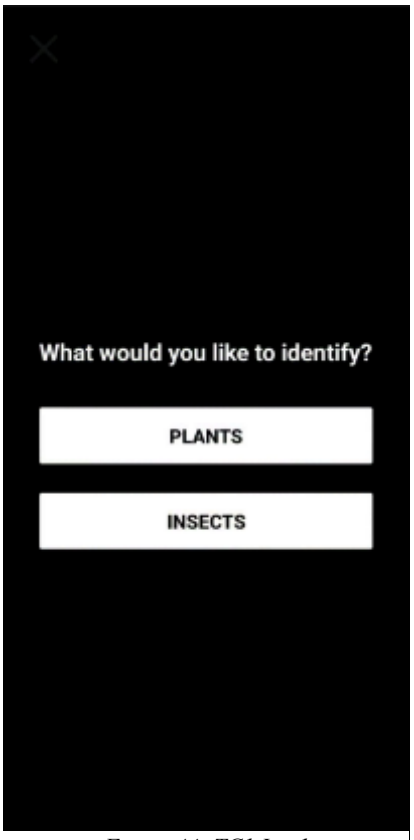


Figure 44: TC1 Img1

Step 2:

Tap on PLANTS ✓

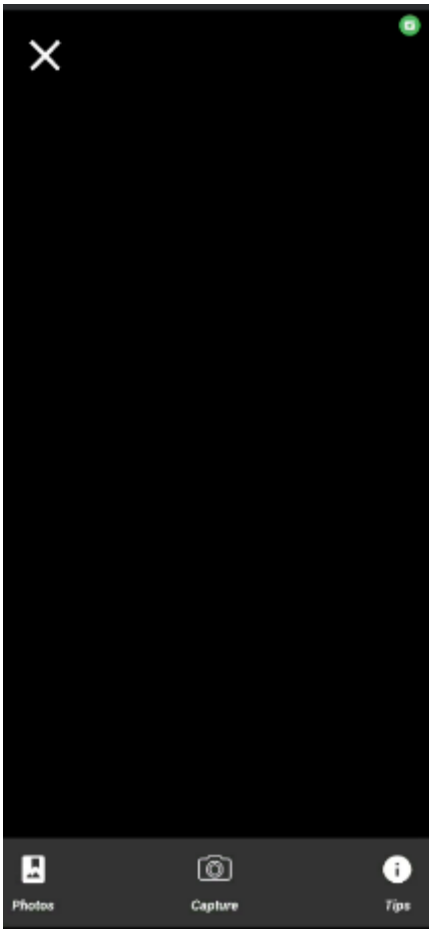


Figure 45: TC1 Img2

Step 3:

Snap a picture of an Insect ✓

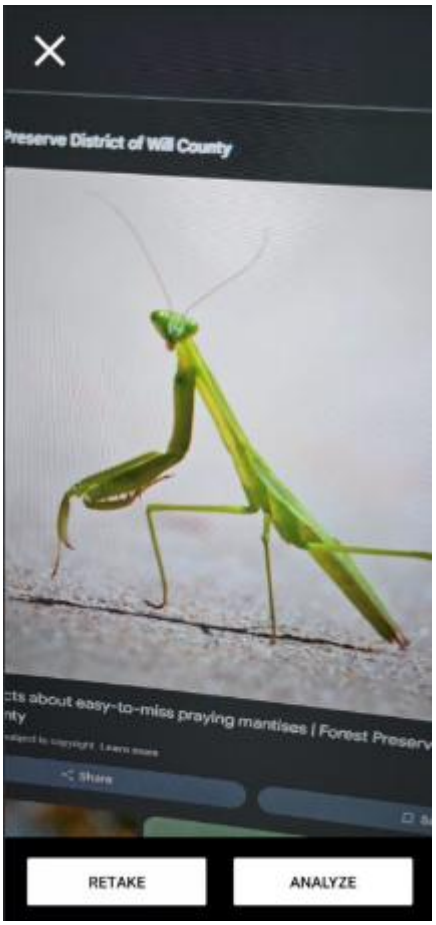


Figure 46: TC1 Img3

Step 4:

Tap on analyse ✓



Figure 47: TC1 Img4

Step 5:

Error should occur ✓

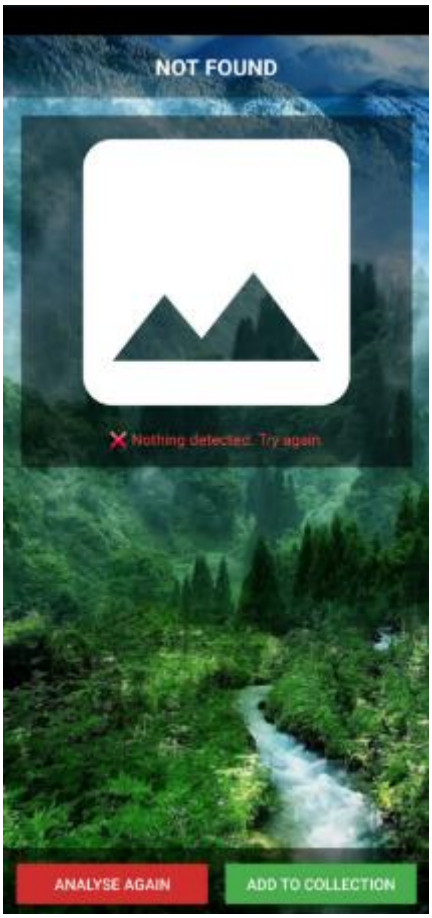


Figure 48:TC1 Img5

4.2 – Testcase 2: Identification History Filtering

Scenario:

We need to test and see whether the identification history works by filtering between insects and plants.

Expected results:

The filtering between plants and insects should work where when plants is selected, only the previous plant identification would be shown. If insects are shown, the previous plant identifications should not be shown.

Steps:

Here are the steps taken to do this testcase.

Step 1:

Go to Profile ✓

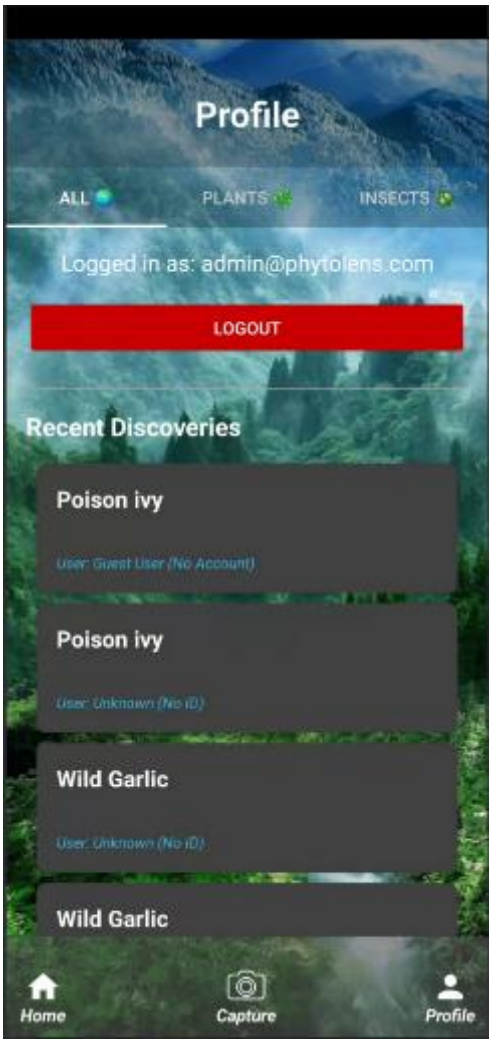


Figure 49: TC2 Img1

Step 2: Tap on plants and only plants should be shown ✓



Figure 50:TC2 Img2

Step 3: Tap on Insects and only Insects should be shown ✓

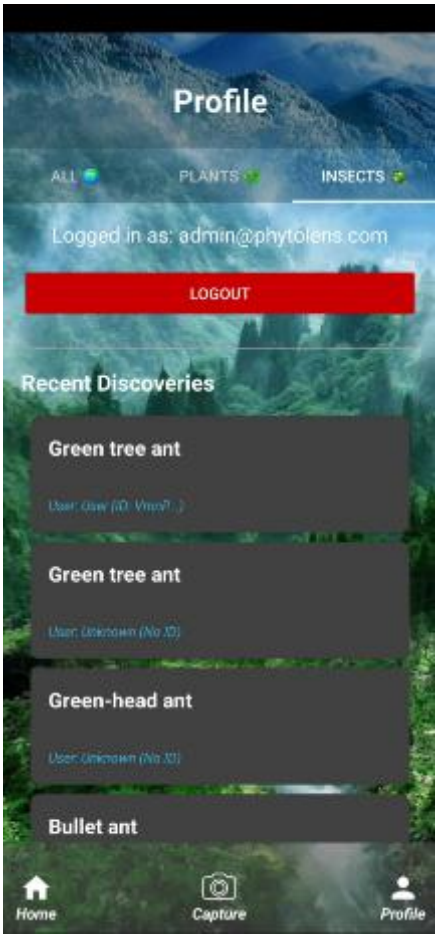


Figure 51:TC2 Img3

4.3 – Testcase 3: Lifecycle resetting

Scenario:

There is a need to test whether the buttons and UI reappear after clicking on the “Analyse Again” button after analysing an image.

Expected Results:

The camera activity screen should reappear and allows for the user to choose between Plant or animal for analysis

Steps:

Here are the steps taken to do this testcase.

Step 1: Snap or choose an image from the gallery and then analyse ✓

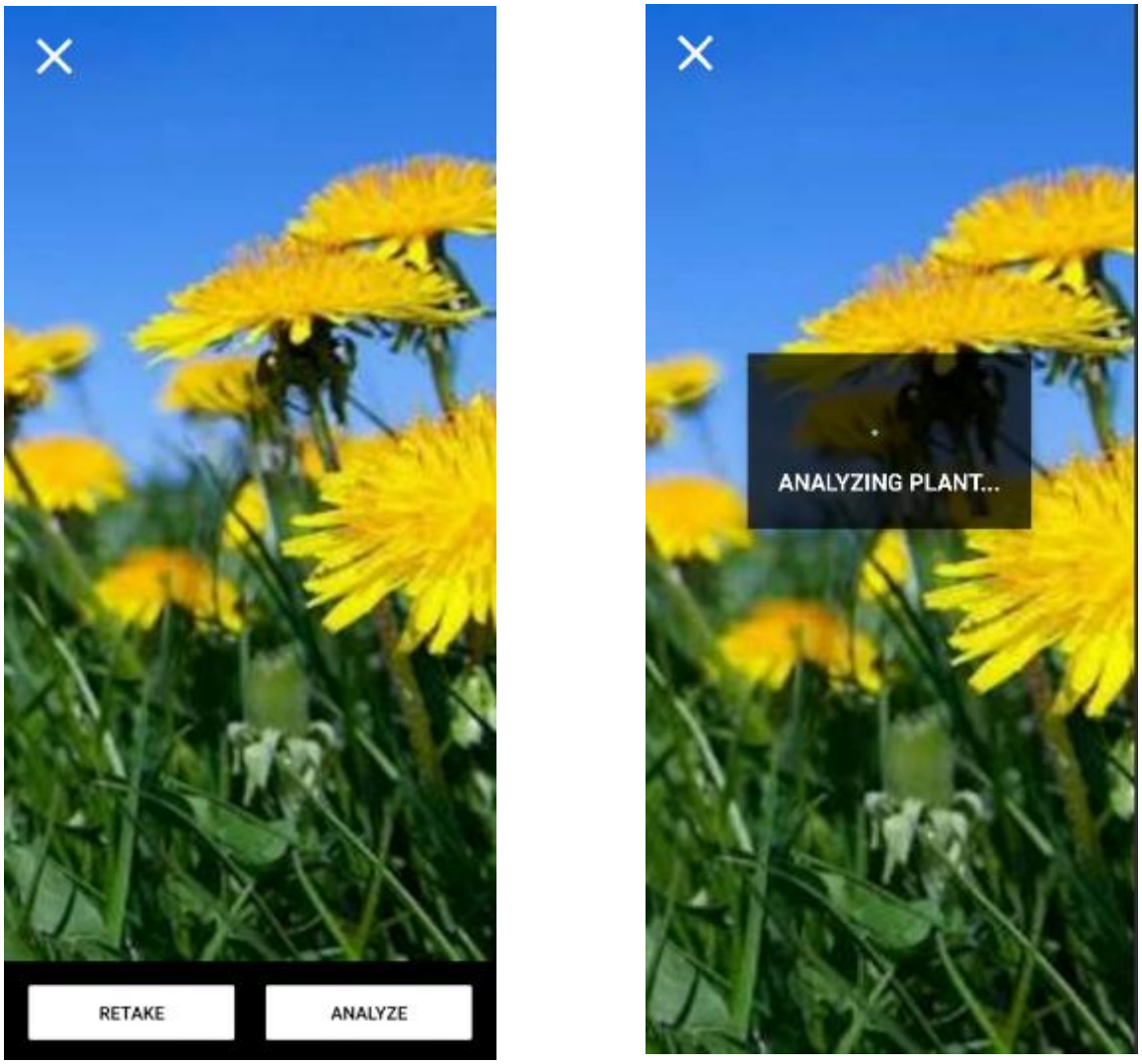


Figure 52: TC3 Img1 and 2

Step 2: User should be brought to the Results screen of after analysis. The “Analyse Again” and “Add to Collection” buttons should appear at the bottom of the screen. ✓



Figure 53: TC3 Img3

Step 3: Click on the “Analyse Again” button and the user should be redirected back to the Camera Activity ✓



Figure 54: TC3 Img4

Step 4: Choose a category to identify. We shall choose plant for now. The UI should all be present ✓



Figure 55: TC3 Img5

4.4 – Testcase 4: Cloud Routing

Scenario:

In this test case, we will discover whether the data is indeed being saved to the database based on the user

Expected results:

The analysed plant shall be saved to the collections in Firebase based on the user who added the analysis to the collection

Steps:

Here are the steps taken to do this testcase.

Step 1: Snap or select an image from the gallery to analyse and then click on analyse. ✓





Figure 56: TC4 Img1 and 2

Step 2: After analysis tap the “Add to Collection” button ✓



Figure 57: TC4 Img3

Step 3:When taken to Analysis history, should show the recently identified plant. ✓

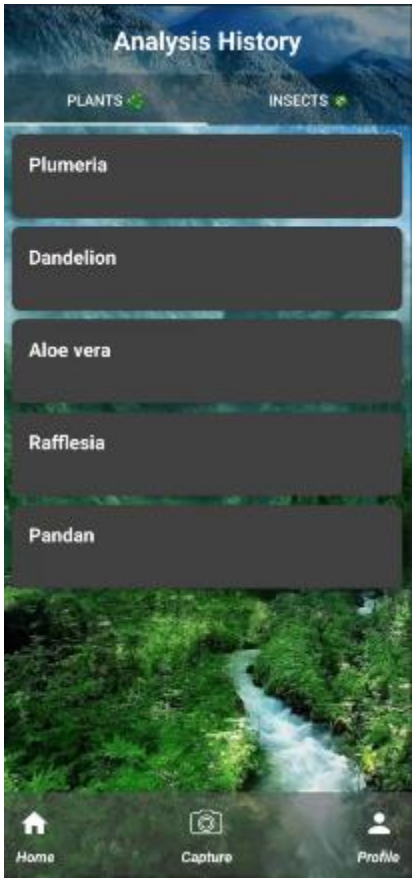




Figure 58: TC4 Img4 and 5

4.5 – Testcase 5: Data Privacy

Scenario:

In this scenario, we are going to test whether the logged in user can see other user's analysis.

Expected results:

The user's previous analysis should appear only and not other user's analysis. This is for data privacy of the users

Steps:

Here are the steps taken to do this testcase.

Step 1: We login as one user. We shall use account name test1 for this part ✓

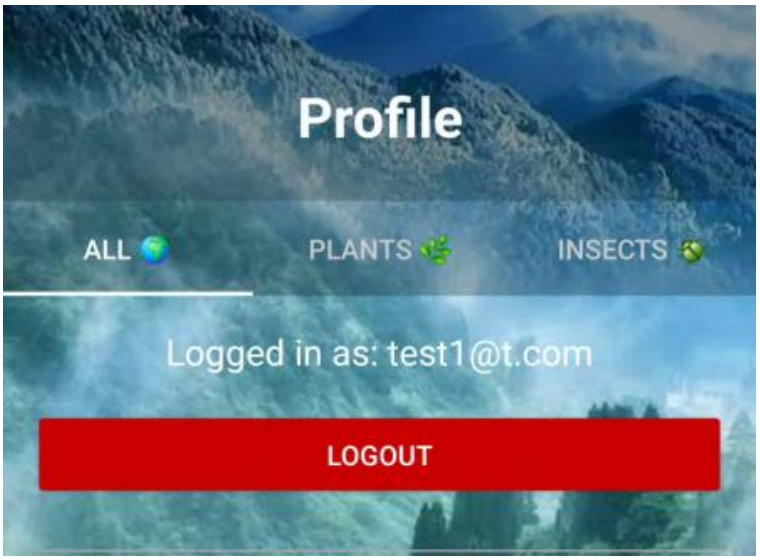


Figure 59: TC5 Img1

Step 2: Tap on profile and their previous analysis should appear. ✓

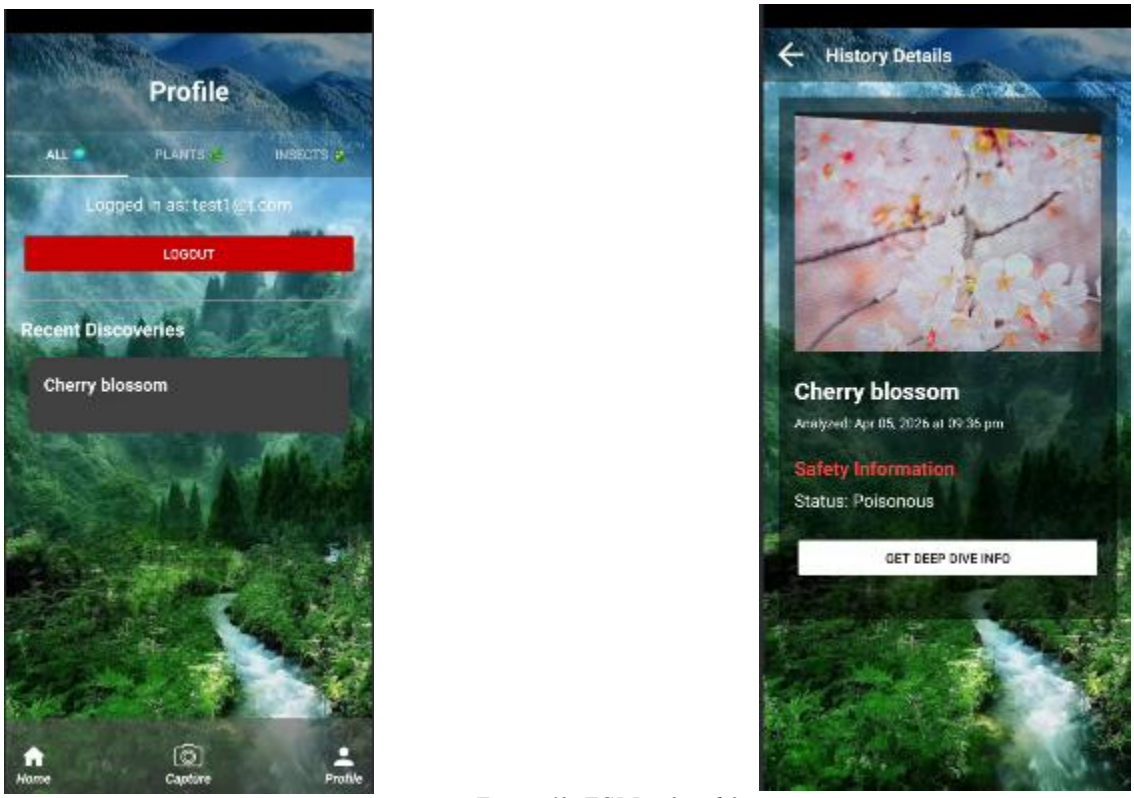


Figure 60: TC5 Img2 and 3

Step 3: Log in as another user, test2 ✓

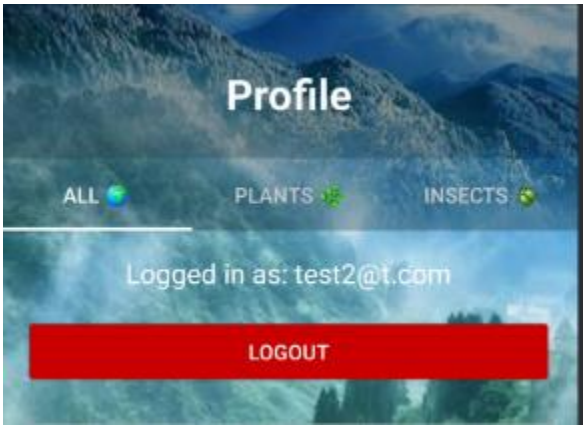


Figure 61: TC5 Img4

Step 4: Go to Profile and user1's data should not be seen ✓

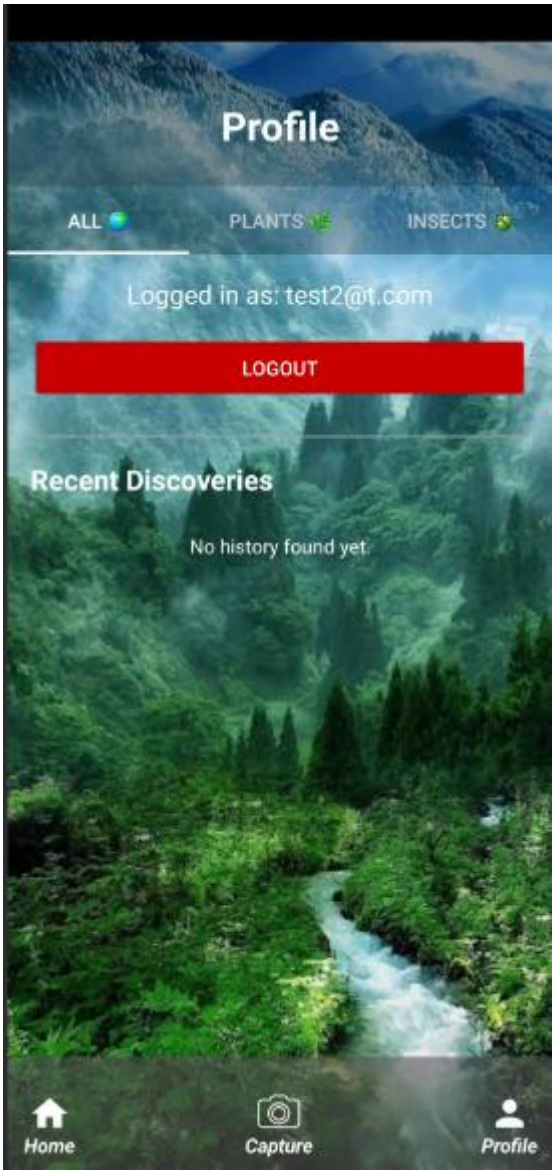


Figure 62: TC5 Img5

4.6 – Testcase 6: Admin Access

Scenario:

In this scenario, we are going to see whether the admin account have elevated privileges like looking at other user's analysis

Expected results:

We should be able to see other user's recent analysis and see the users who conducted that analysis.

Steps:

Here are the steps taken to do this testcase.

Step 1: Login into the admin account ✓

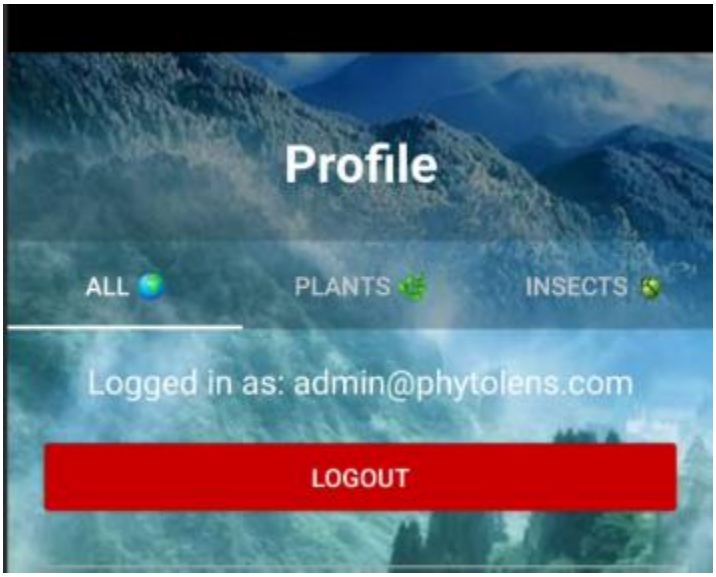


Figure 63: TC6 Img1

Step 2: Proceed to the Profile. There we should be able to see all the previous analysis done by all the users of the application ✓

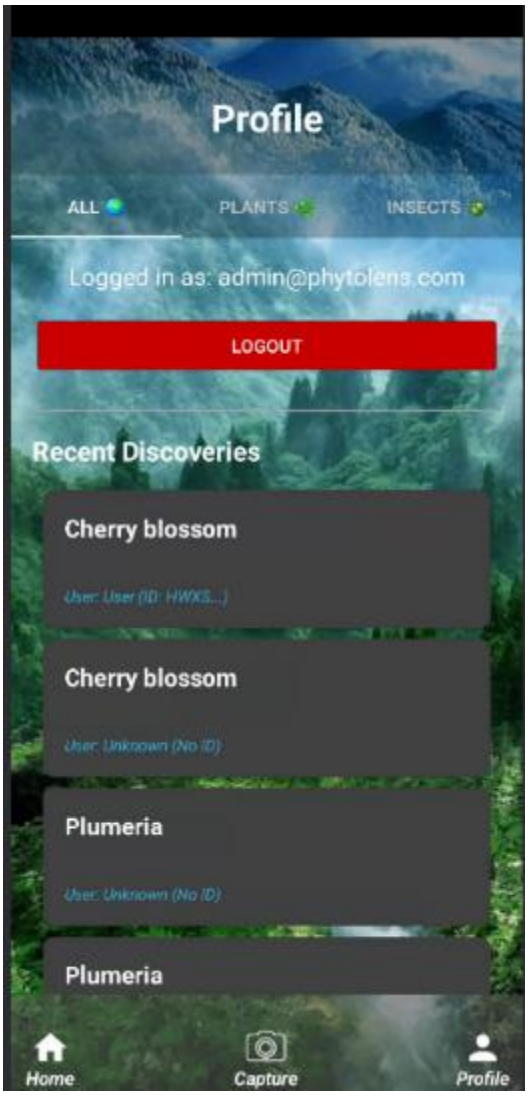


Figure 64: TC6 Img2

Testcase Result Status

Testcase 1: Passed

Testcase 2: Passed

Testcase 3: Passed

Testcase 4: Passed

Testcase 5: Passed

Testcase 6: Passed

Chapter 5 – Conclusion & Future work

5.1 – Conclusion

After two whole semesters of constant R&D and constant testing, PhytoLens has managed to accomplish its main objectives which is to identify plants and insects. In the second half of the project is when I really hit it off because way much more was achieved while doing the project. Things like the robust identification of plants and the implementation of the insect identification tool was done in the second half.

By integrating Gemini's API, this allowed the AI to perform quick image to text analysis. The AI had managed to analyse and give relevant information (Name, Toxicity Status, Fun Facts, Recipes) about 99% of the time which is assuring. Furthermore, with the backend being handled by Firebase, this ensures that the data is secured properly in a robust DB. The use of Firebase had allowed me to easily integrate admin and normal user's access.

While current technological limitations had not allowed me to implement the use of the AI offline, I feel that this would be possible in the future since 6G is coming soon and the use of satellite for internet connectivity is something that is coming in the foreseeable future.

5.2 – Recommendation in Future Work

5.2.1 – Offline Mode

For the future work, I want to really implement the offline mode as I feel that this would be the selling point of the application since no known AI out there can be used in offline mode because the size of the application would be huge since it's an LLM. I would like to implement this the moment a new AI model that has offline capabilities and do not take a lot of space to use.

5.2.2 – Using AR for the PhytoLens

A future work that I can possibly do is using Live Lens which is using AR to highlight poisonous plants in the area or beneficial herbs in the area as the user scans their environment. I feel that this would increase the chances of survival if the user were stranded

in the wilderness. This would enhance the user's experience further since they can just use real-time scanning rather than using a single photo every time to analyse an unknown insect or plant.

5.2.3 – Satellite Connectivity

I can try to implement this when the 6G technology has been rolled out. With the current technology, 4G and 5G networks have limited connectivity if a user is deep within the wilderness. With the help of 6G, PhytoLens can remain fully functional anywhere on Earth which guarantees the safety of explorers.

5.2.4 – Social Verification of analysis

By implementing this, I can get community feedback regarding analysis of plants or insects by upvotes or downvotes or verification by experts and biologists. This would help to increase the scientific reliability of the application

5.2.5 – Gamification and Challenges

I would really like to implement this as this would add a fun touch to the application and would make people use the app for rewards and challenges. This can help to encourage community engagement and help to promote a healthy lifestyle. This feature I can add monthly, weekly and daily challenges like finding 3 species of ferns or find 5 types of ants. The rewards for these challenges would be things like badges and points that could be exchanged for vouchers. This gamification of the PhytoLens would motivate users to leave their homes and touch grass by entering the actual reality. This would improve their physical well being by exploring the outdoors.

Reflection on Learning Outcome Attainment

Throughout this project, I have encountered various hurdles that required me to use my analytical skills to resolve. One example is the latency and the timeout issues when doing the deep dive analysis to get more information of an analysis. How I did it was that I looked through the Log CAT in Android Studio and found out the problems by googling the timeout codes and then finding out through articles online and found out that I needed to add a delay so that the AI have time to think and analyse. So, I added a 60 second delay so that the AI has sufficient time to process the request and then get the accurate information to display.

On top of the hurdles, I have successfully learnt how to make a full-fledged mobile application that bridges the mobile hardware which is the smartphone and cloud-based AI which is Gemini. Initially I made an all-in-one analyser but realised that logically its not right so I have decided to split the plants and insects' analysis. Each of them has differing AI prompts depending on the mode that was chosen. This method ensures that the data is store properly and systematically in the database (Firebase) and the UI remains more relevant by providing useful information like the toxicity and the name of the plant / insect before deep diving.

For the lifelong learning, I felt that my whole journey in developing PhytoLens has taught me that AI is a rapidly evolving and demanding things. This project made me realise the importance of staying updated with the available technology so that I can make use of the latest models which are usually more efficient. My experience gained while debugging problems and issues in the applications has taught me patience as it was rather frustrating since there were so many syncing and layout issues. These are skills that I feel is needed in the future when I go out to work as I would be on my own and no professors or lecturers would be there to help me should I face any issues with my work.

References

[1] Top 10 Mobile App Development Frameworks in 2025, GeeksForGeeks 15th July 2025, [Online]. Available: https://www.geeksforgeeks.org/blogs/top-mobile-app-development-frameworks/

[2] Android SDK and it's Components, GeeksForGeeks, 6th January 2025, [Online]. Available: https://www.geeksforgeeks.org/android/android-sdk-and-its-components/

[3] Google AI results from google search. Prompt given: "OpenAI tools for image recognition for mobile app development"[Online]. Available: https://www.google.com/search?q=openai+tools+for+image+recognition+for+mobile+app+development&sca_esv=fa0b78d7cd41bbf7&sxsrf=AE3TifM2pPT19g1gz21cb5VqU2GvxYapUQ%3A1762541036703&ei=7D0OadK2KpjuseMPqb274Qc&oq=openAi+tools+for+image+recog&gs_lp=Egxnd3Mtd2l6LXNlcnAiHG9wZW5BaSB0b29scyBmb3IgaW1hZ2UgcmVjb2cqAggBMgUQIRigATIFECEYoAEyBRAhGJ8FMgUQIRifBUj1JlDuBVi8F3ACeAGQAQCYAU6gAawFqgECMTK4AQHIAQD4AQGYAg6gAtMFwgIKEAAYsAMY1gQYR8ICBhAAGBYYHsICCxAAGIAEGIYDGIoFwgIIEAAYogQYiQXCAggQABiABBiiBMICBRAAGO8FwgIHECEYoAEYCpgDAIgGAZAGCJIHAjE0oAftQ7IHAjEyuAfOBcIHBjEuMTIuMcgHGg&sclient=gws-wiz-serp

[4] Image understanding, Gemini API. [Online]. Available: https://ai.google.dev/gemini-api/docs/image-understanding

[5] Top 7 Databases for Mobile App Development in 2025, GeeksForGeeks, 23rd July 2025 [Online]. Available: https://www.geeksforgeeks.org/android/databases-for-mobile-app-development/